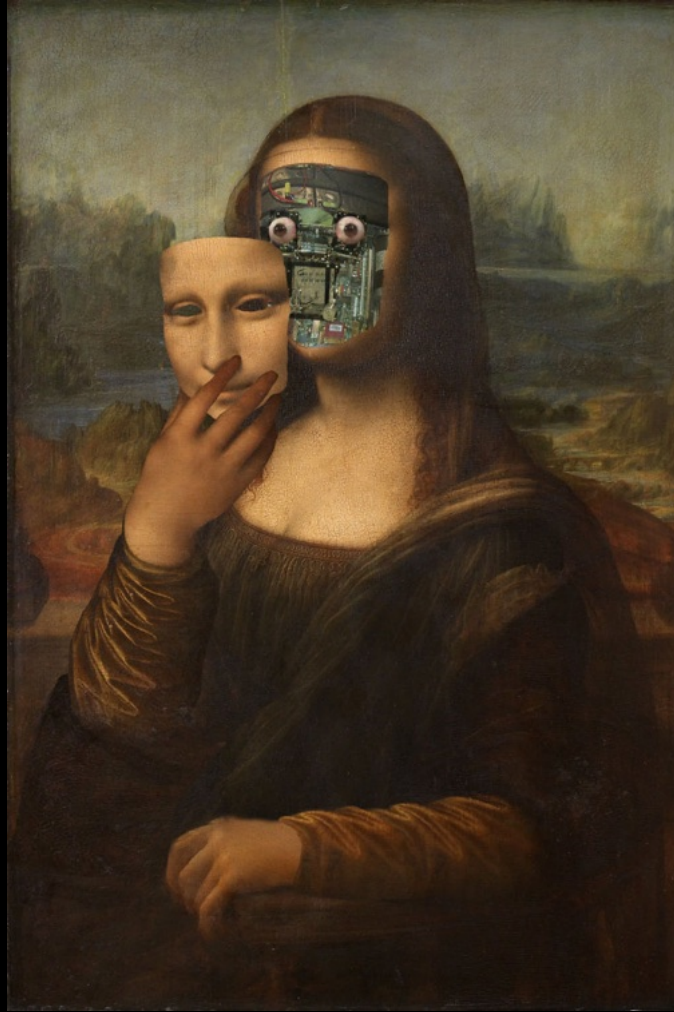


Dezvoltarea aplicațiilor Web



ingineria specificării cunoștințelor

Prof. Sabin Corneliu Buraga

profs.info.uaic.ro/sabin.buraga/

*“It is not the strongest of the species
that survive, nor the most intelligent,
but the ones most responsive to change.”*

Charles Darwin



Există metode de proiectare (modelare)
a cunoștințelor (ontologiilor)?

managementul cunoștințelor

Aspect de interes:
crearea de ontologii din diverse surse

scheme XML

scheme de baze de date

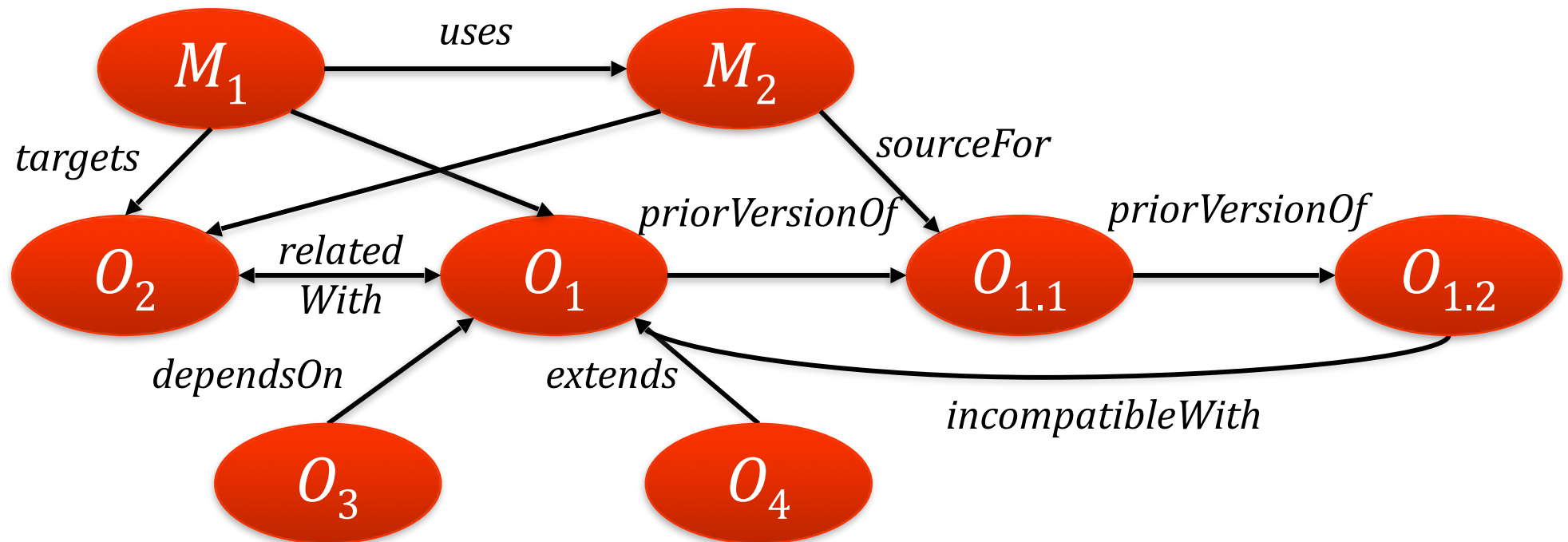
diagrame UML

foi de calcul tabelar

altele – *e.g.*, CSV, microformate, microdate HTML5,...

managementul cunoștințelor

Aspect de interes:
gestiunea versiunilor



managementul cunoștințelor

Aspect de interes:

constituirea/folosirea colecțiilor de ontologii

asocieri

comparații

reconcilieri

validări

conversii

asigurarea calității

managementul cunoștințelor

Aspect de interes:

modularizarea ontologiilor

„împrumutarea” entităților specificate de ontologii

managementul cunoștințelor

Aspect de interes:

modularizarea ontologiilor

„împrumutarea” entităților specificate de ontologii

facilitează dezvoltarea colaborativă + reutilizarea

V. Presutti, *Methods for Ontology Design Patterns Reuse*, ISWC 2016
www.slideshare.net/vpresutti/methods-for-ontology-design-patterns-reuse

seria WoMO (*Workshops on Modular Ontologies*): iaoa.org/womo/



Datasets



Cruises



Vessels



Instruments



Physical Samples



Gazetteer Feature



Researchers



Organizations



Awards



exemplu practic vizând cunoștințe geografice:
Pascal Hitzler, *Modular Ontology Design and Use Case:
The GeoLink Example*, ESWC 2017

managementul cunoștințelor

Aspect de interes:

obținerea, organizarea și vizualizarea
domeniului de cunoaștere
înainte și în timpul creării unei ontologii

managementul cunoștințelor

Aspect de interes:

utilizarea ontologiilor în funcție de context

axate asupra modelării – exhaustive – a unei lumi (*web*)

specifice unui set de *task*-uri/procese – *workflow*-uri

mixte

necesitate

Metodologii de inginerie a cunoștințelor
(*knowledge engineering methodologies*)

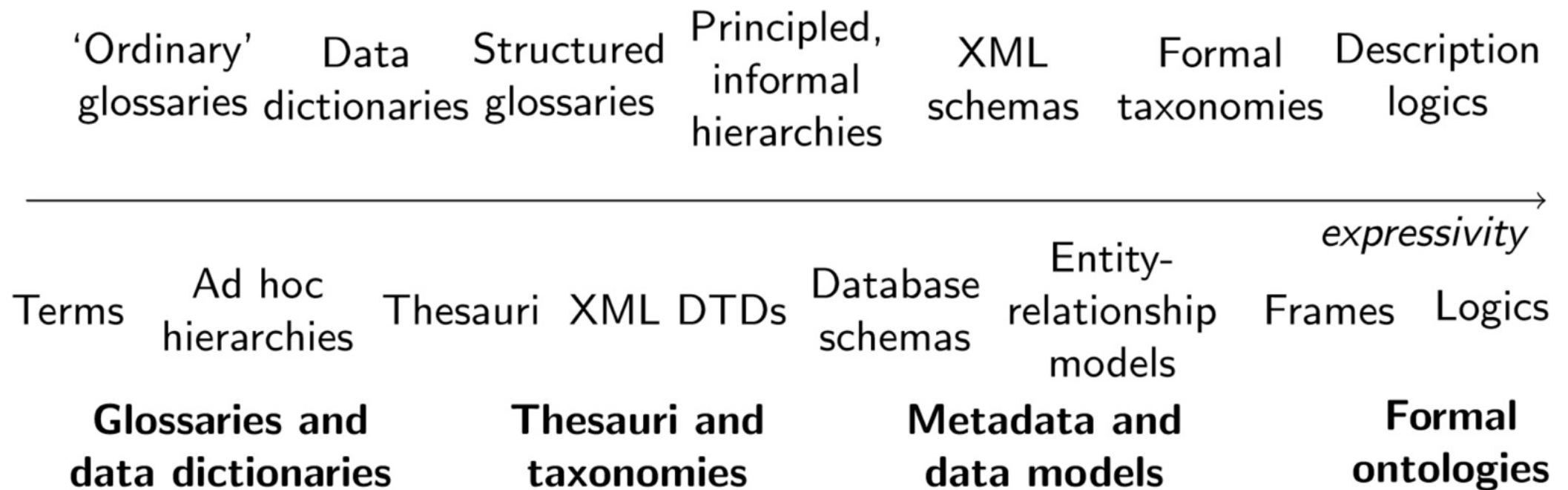
procesul prin care **experți ai domeniului** și **ontologiști**
vor constitui o **graf/bază de cunoștințe**, modelat(ă)
via un **limbaj de reprezentare a cunoștințelor**
și gestionat(ă) cu o **suită de instrumente** adiționale

C. Maria Keet, *An Introduction to Ontology Engineering*, 2020
people.cs.uct.ac.za/~mkeet/OEbook/

necesitate

Metodologii de inginerie a cunoștințelor
(*knowledge engineering methodologies*)

procesele, limbajele și instrumentele folosite se bazează
pe diverse paradigme de reprezentare a cunoștințelor



maniere diferite de specificare a cunoștințelor
(Jérôme Euzenat & Pavel Shvaiko, 2014)

necesitate

Metodologii de inginerie a cunoștințelor
(*knowledge engineering methodologies*)

pot recurge la abordări de tip *Model-Driven Engineering*

de (re)văzut **MDA** – *Model Driven Architecture*

abordări agile

RapidOWL (Sören Auer & Heinrich Herre, 2007)

SAMOD – *Simplified Agile Methodology
for Ontology Development* (Silvio Peroni, 2016)
essepuntato.it/papers/samod-owled2016.html

Eva Blomqvist, Karl Hammar, Valentina Presutti,
“Engineering Ontologies with Patterns: The eXtreme Design
Methodology”, *Ontology Engineering with Ontology Design
Patterns*, IOS Press (2016)
karlhammar.com/downloads/blomqvist2016engineering.pdf



Care sunt procesele de modelare ontologică?

observații

Dezvoltarea unei ontologii implică
partajarea de informații
+
colaborare

observații

“Knowledge is fractal” (Alan Rector)

ontologiile prezintă aceeași structură
la fiecare nivel de granularitate (detaliu)

observații

Atenție la dezvoltările exponențiale ale ontologiei

*“If you wish to make an apple pie from scratch,
you must first invent the universe.”*

Carl Sagan

complicații

Necunoașterea *a-priori* a dimensiunii ontologiei

Numărul persoanelor implicate

Folosirea ontologiei de către utilizatori
care nu sunt experți ai domeniului modelat

Natural laziness 🤔

...

inginerie: dificultăți

Conflicte privitoare la intuiții

intuițiile sunt dificil de formalizat

experții în domeniu sunt conduși de obisnuințe
și practică (șabloane de proiectare)

► prototipuri + generalizări

inginerie: dificultăți

Conflicte privitoare la intuiții

intuițiile sunt dificil de formalizat

logicienii sunt motivați de logică și computabilitate

- ▶ definiții/axiome (formale) + operatori universali

inginerie: dificultăți

transparență și predictibilitate

versus

rigoare + completitudine

inginerie: dificultăți

Persoanele

- ontologiști, programatori, utilizatori –
fără (prea multă) experiență

inginerie: dificultăți

Studiu de caz: alergia la anumite substanțe
(Peter Morville)

acces urgent la informații exacte și utile
credibilitatea surselor este esențială

Q /food/cheese		Xin Luna Dong (2016)		
Site	Accuracy	# Triples	Score	▼
en.wikipedia.org	0.76	1,184	5.36	
www.cheesewiki.com	0.84	230	4.55	
www.ranker.com	0.93	105	4.34	
scratchpad.wikia.com	0.92	101	4.26	
www.marcellathecheesemonger.com	0.90	109	4.25	
www.cheeseplatesf.com	0.90	114	4.25	
cheesetique.com	0.93	83	4.12	
www.lafromagerie.co.uk	0.93	62	3.87	

inginerie: dificultăți

Studiu de caz: alergia la anumite substanțe

orice căutare folosind un motor de căutare clasic eșuează
popularitate $\neq$ autoritate în domeniu

cunoștințele (relevante) trebuie să fie regăsite pe Web
necesită expertiză în domeniu +
abilități avansate de căutare pe Web

“dialogul” cu un sistem de tip întrebare-răspuns poate
conduce la obținerea unor rezultate neconcludente/eronate

inginerie: dificultăți

Modelarea

semantica (*meaning*)

corectitudinea clasificărilor și extragerii cunoștințelor

inginerie: dificultăți

Modelarea

indexarea

vizează activitățile de descoperire,
căutare și găsire a cunoștințelor

inginerie: dificultăți

Modelarea

popularea

introducerea datelor (*e.g.*, indivizi)

+

procesul decizional

eventual, alte aspecte de interes

inginerie: dificultăți

Modelarea

achiziția

capt(ur)area cunoștințelor

inginerie: dificultăți

Asigurarea calității + managementul modificărilor

calitatea

stabilirea de criterii privitoare la ceea ce este „corect”
(*e.g.*, grad de expresivitate, ușurința refolosirii,
volumul entităților modelate: clase/proprietăți/restricții/
indivizi,...)

inginerie: dificultăți

Asigurarea calității + managementul modificărilor

evoluția

luarea în calcul a schimbărilor viitoare ale ontologiei

inginerie: dificultăți

Asigurarea calității + managementul modificărilor

testarea

controlul modificărilor

asigurarea – eventual, (semi-)automată – a calității
de exemplu, detectarea inconsistențelor

inginerie: dificultăți

Privitoare la factorul uman

apariția confuziilor privitoare la termeni
și la utilizarea ontologiei în practică

inginerie: dificultăți

Privitoare la factorul uman

persoane provenind din medii eterogene

inteligență artificială, interacțiune om-calculator, dezvoltare

Web, inginerie software,

lingvistică (computațională), științe cognitive, filosofie,

...

inginerie: dificultăți

Referitoare la interoperabilitate

recurgerea la sintaxe diferite

utilizarea de instrumente de modelare eterogene

folosirea unor limbaje având diverse expresivități

inginerie: dificultăți

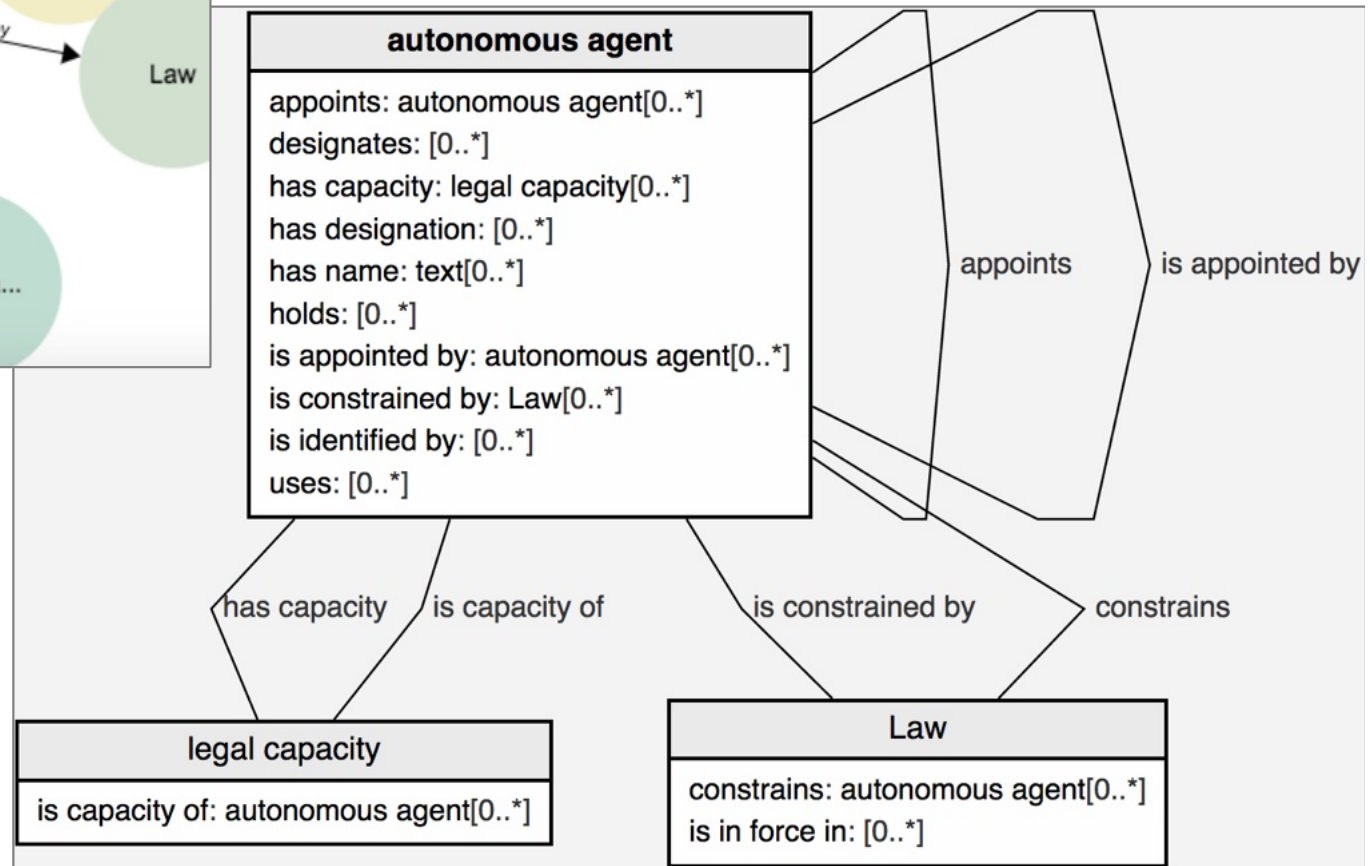
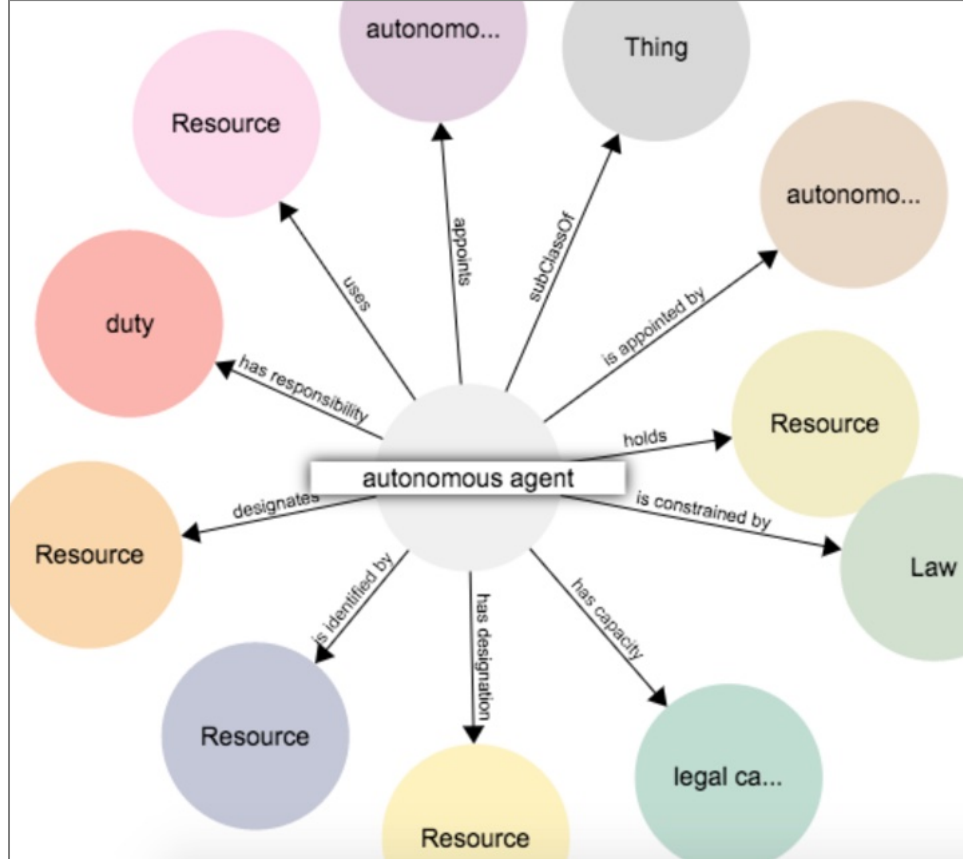
Vizualizarea (redarea) ontologiilor

prezentări alternative

documentare automată

navigare bazată pe context + *faceted search*

aspect important: ontologiile de mari dimensiuni

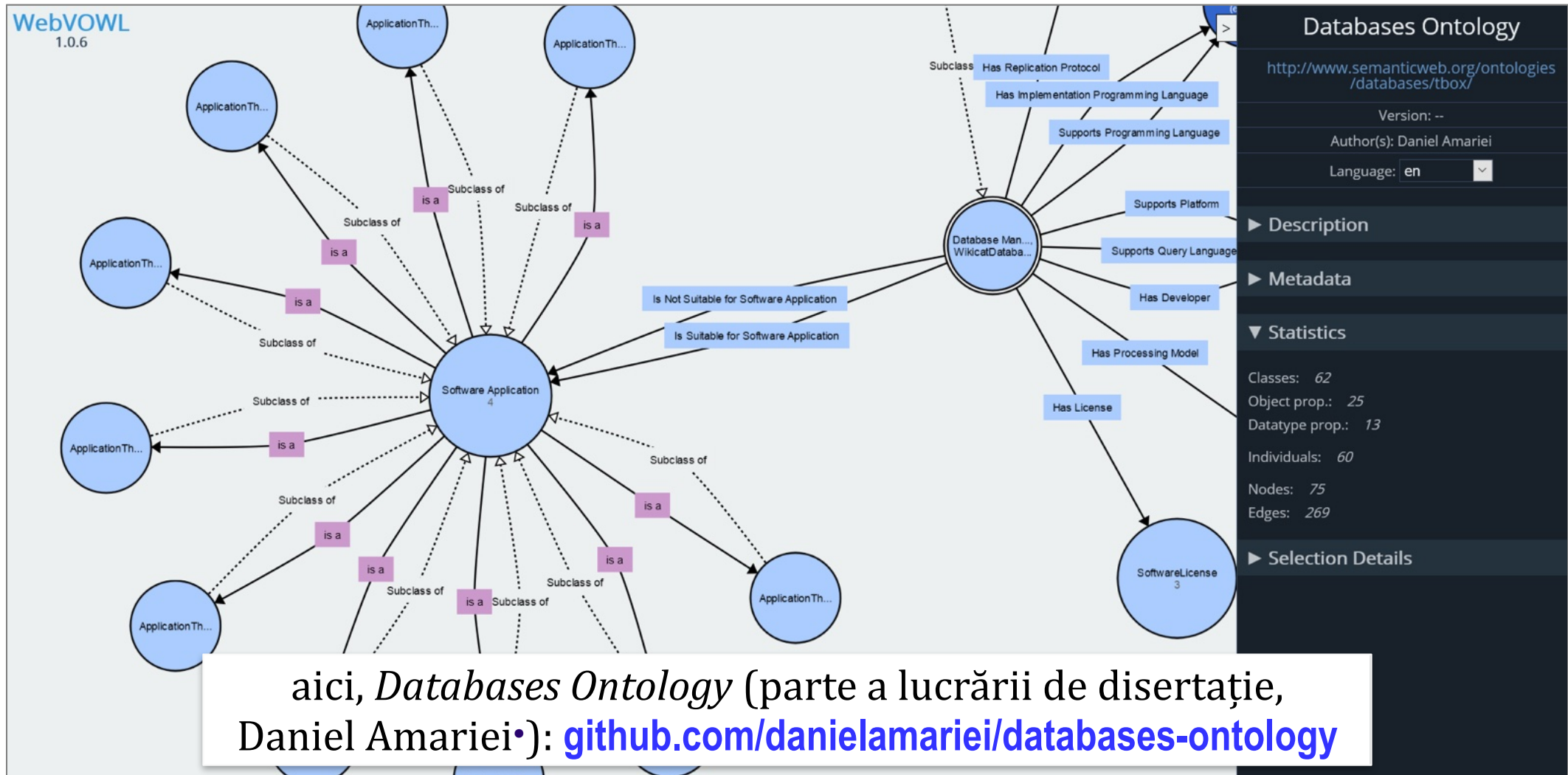


diverse vizualizări ale unei ontologii
 cu instrumentul **TopBraid EDG Vocabulary Management**

VOWL – *Visual Notation for OWL Ontologies*

reprezentarea grafică a elementelor unei specificații OWL
(Steffen Lohmann, Ștefan Negru•, Florian Haag, Thomas Ertl, 2016)

github.com/VisualDataWeb/WebVOWL

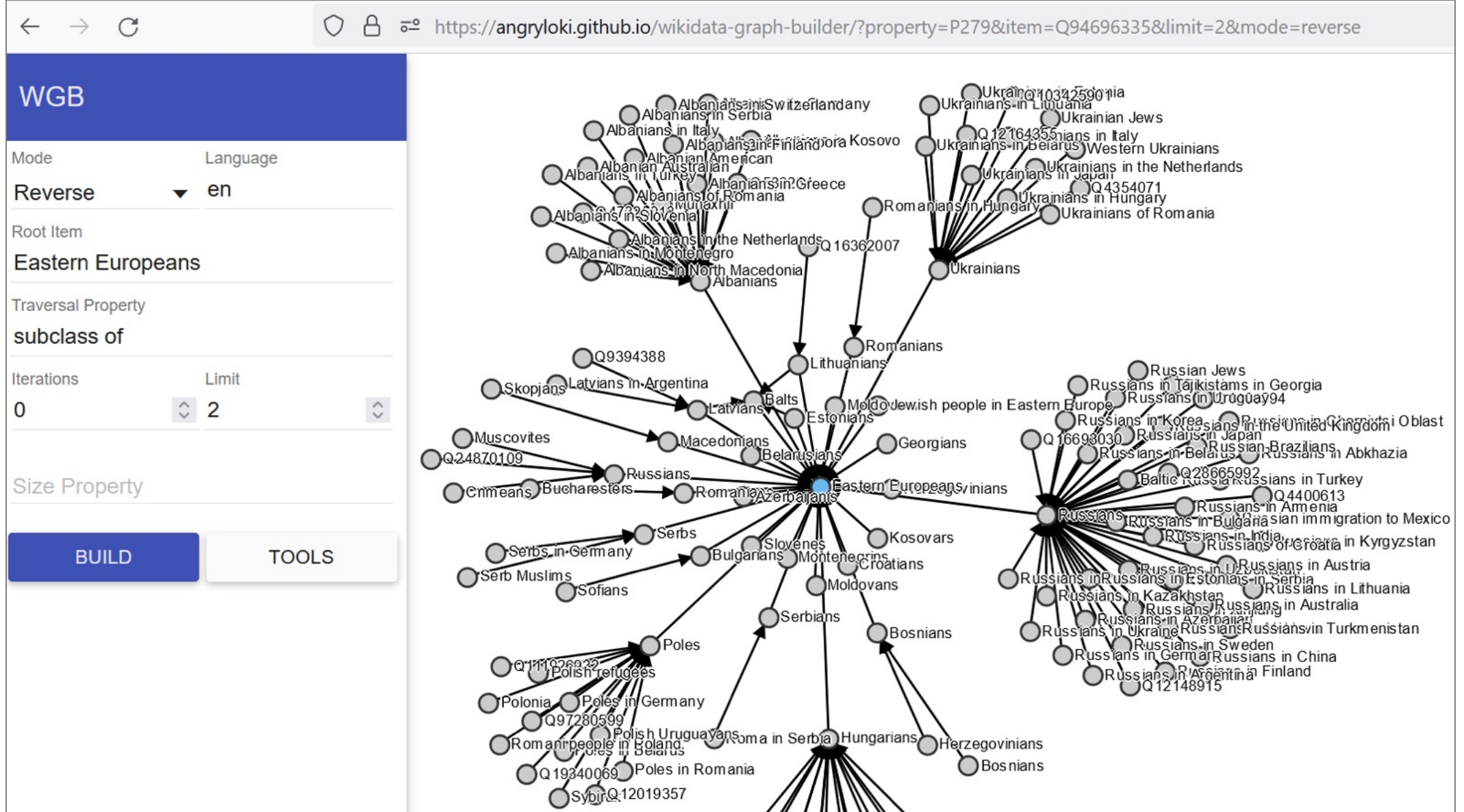


inginerie: dificultăți

Vizualizarea (redarea) ontologiilor

descoperirea datelor și relațiilor (*data/relation mining*)

eventual, folosind instrumente de realizare
a raționamentelor automate (*reasoners*)



găsirea de relații între entitățile oferite de Wikidata:
instrumentul **Wikidata Graph Builder**
angryloki.github.io/wikidata-graph-builder/

inginerie: realități

Alegerea dificilă a unor criterii formale
(*i.e.* privind calitatea cunoștințelor modelate)

consistență
completitudine
concizie
expresivitate

...

inginerie: realități

Generarea manuală a ontologiilor

taxonomii navigabile – *e.g.*, Curlie

folksonomii – vezi sistemele de *tagging* social

ontologii publice – *e.g.*, DBpedia Ontology, FoodON, KBpedia, Music Ontology, schema.org, YAGO,...

ontologii interne (proprietary)

inginerie: realități

Ontologiile generale (*top-level*)
nu sunt cu adevărat utile în practică

inginerie: realități

Apar probleme teoretice/practice privind:

asigurarea coerenței

facilitarea procesării

folosirea surselor de date multiple

inginerie: realități

Diferite vocabulare în vigoare

clasa $\approx$ concept $\approx$ categorie $\approx$ tip

instanță $\approx$ individ

entitate $\approx$ obiect (clasă/individ)

proprietate $\approx$ *slot* $\approx$ relație $\approx$ atribut $\approx$
 $\approx$ rol $\approx$ legătură semantică

inginerie: realități

Sintaxe diverse existente

sintaxa abstractă – vezi OWL Manchester

N3 (Turtle)

XML/RDF

JSON-LD

logica de ordin I

formatul Protégé

limbajul natural (parafraza)

...

metodologie (Noy & McGuinness)

1. Stabilirea domeniului de cunoaștere
2. Organizarea ontologiei
3. Popularea („umplerea”) ontologiei
4. Verificarea ontologiei
5. Publicarea ontologiei

metodologie

Stabilirea domeniului de cunoaștere

asamblarea surselor de date și expertiza
în domeniu pentru a constitui concepte (*things*)

se vor asigura – măcar informal – consensul și consistența

metodologie

Organizarea ontologiei

proiectarea pe ansamblu a structurii conceptuale

identificarea principalelor concepte concrete

și a proprietăților acestora

identificarea relațiilor dintre concepte

crearea conceptelor abstracte (de nivel înalt)

identificarea instanțelor

referențierea/includerea altor ontologii

...

metodologie

Popularea ontologiei

adăugarea conceptelor

inserarea relațiilor + constrângerilor/axiomelor

adăugarea indivizilor

...la nivelul de detaliere necesar
pentru realizarea scopurilor ontologiei

metodologie

Verificarea ontologiei

reconcilierea inconsistențelor dintre elemente,
la nivel sintactic, logic și semantic

metodologie

Verificarea ontologiei

reconcilierea inconsistențelor dintre elemente,
la nivel sintactic, logic și semantic

verificarea consistenței poate implica
o clasificare automată, rezultând noi concepte bazate
pe proprietăți individuale sau relații între clase

metodologie

Publicarea ontologiei

verificarea finală de către experți ai domeniului

publicarea + utilizarea efectivă
în cadrul comunității de practică

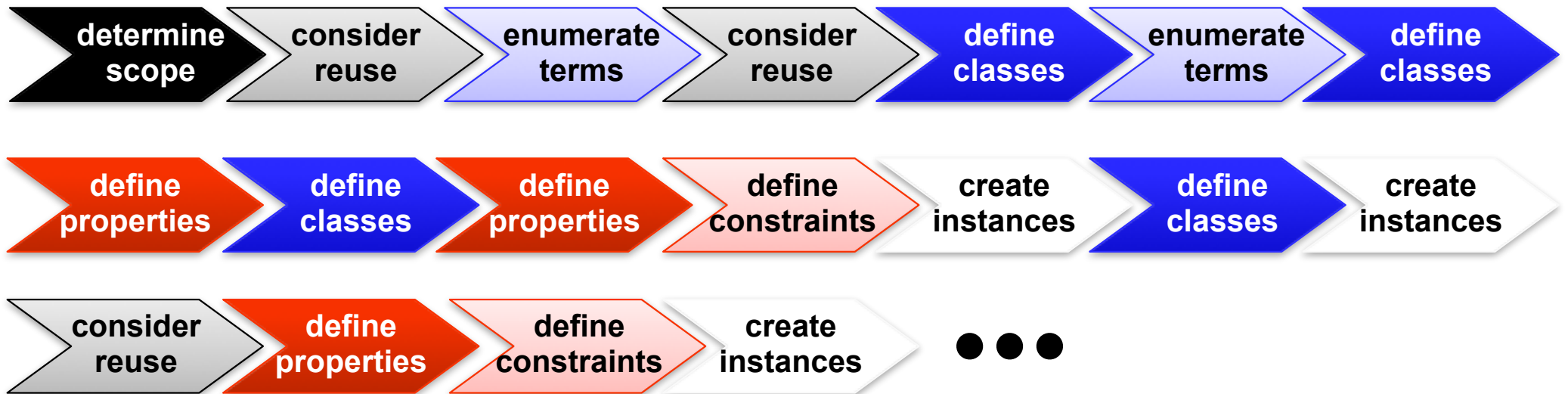
metodologie

Procesul ideal de dezvoltare a unei ontologii



metodologie

Procesul realist de dezvoltare a unei ontologii



metodologie (Rector & Horrocks)

1. Stabilirea scopului
2. Colectarea cunoștințelor
la nivel informal ori semi-formal
3. Rafinarea cerințelor + testarea
4. Implementarea (materializarea modelului) –
modelarea propriu-zisă
5. Evaluarea + asigurarea calității
6. Monitorizarea utilizării și a evoluției

metodologie

Stabilirea scopului

cerințe

evaluare

reutilizare

...

metodologie

Colectarea cunoștințelor
la nivel informal ori semi-formal
stabilirea termenilor

organizarea informală a termenilor

parafrizarea și clarificarea termenilor
▶ definiții informale ale conceptelor

realizarea de diagrame informale

metodologie

Rafinarea cerințelor + testarea

metodologie

Implementarea – modelarea propriu-zisă
parafrizare și comentare la fiecare fază

dezvoltarea unei scheme normalizate și
a unui „schelet” de ontologie

implementarea unui prototip (consemnând intențiile)

verificarea scalabilității

popularea ontologiei – *text mining*, procesare a
limbajului uman, JSON, microdate, RDFa, baze de date,...

metodologie

Evaluarea + asigurarea calității

urmărirea scopului inițial

includerea de verificări privind evoluția ontologiei
și ajustarea manierei de management al ei

proiectarea de teste și „probe”
în mod similar testării aplicațiilor software

metodologie

Monitorizarea utilizării și a evoluției
(mentenanță)

“process not product”

Alan Rector

aspecte importante

Raționamentul lumilor închise vs. deschise

lumi deschise (*open world reasoning*)

negația reprezintă contradicție – orice poate fi adevărat
până când se poate demonstra că e fals

aspecte importante

Raționamentul lumilor închise vs. deschise

lumi închise (*close world reasoning*)

negația înseamnă eșec – orice nu poate fi găsit este fals

aspecte importante

Raționamentul lumilor închise vs. deschise

ontologiile sunt considerate lumi deschise,
iar bazele de date lumi închise

aspecte importante

Knowledge Base vs. Data Base

o bază de cunoștințe nu definește un model unic,
ci reprezintă un set de constrângeri
care pot defini o **mulțime de modele posibile**

aspecte importante

Knowledge Base vs. Data Base

o bază de cunoștințe nu definește un model unic,
ci reprezintă un set de constrângeri
care pot defini o **mulțime de modele posibile**

nicio constrângere ► orice model e posibil

cu cât există mai multe constrângeri,
cu atât numărul modelelor posibile scade

aspecte importante

Knowledge Base vs. Data Base

o baza de date definește un **model unic**
(Bechhofer, 2004)

numele diferite sunt interpretate ca indivizi distincți

domeniul e compus numai din indivizii numiți
în cadrul bazei de date – modelul lumii închise

extensiile sunt cât mai reduse posibil

aspecte importante

Knowledge Base vs. Data Base

model unic

expresivitate redusă – nu există negații/disjuncții

nu poate captura informații incomplete

non-monoton:

adăugarea de noi date nu păstrează adevărul

procesul de *reasoning* (interogare) este facil și rapid

aspecte importante

Knowledge Base vs. Data Base

modele multiple

expresivitate ridicată – conectori booleeni, reuniune,...

se pot captura/exprima informații incomplete

monotonie – adăugarea de noi date păstrează adevărul

procesul de *reasoning* (interogare) este dificil și lent

aspecte importante

Activitatea de reasoning

poate fi realizată automat de un *reasoner*

clasificarea conceptelor

rezultă:

un arbore – ierarhie strictă, taxonomie

sau

un digraf aciclic (DAG) – poli-ierarhie, graf de concepte

aspecte importante

Activitatea de reasoning
poate fi realizată automat de un *reasoner*

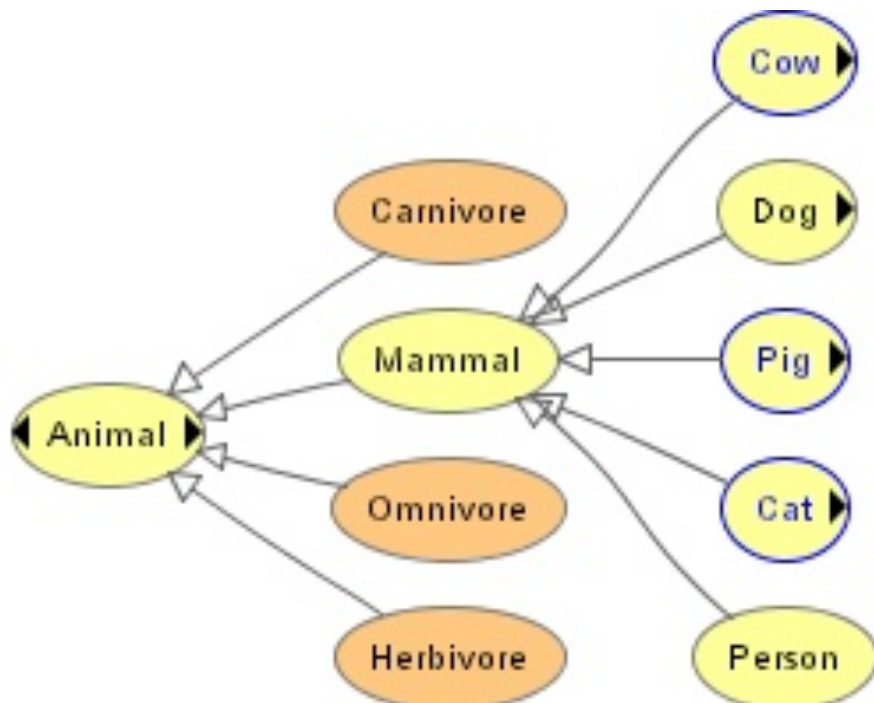
normalizare

separarea *conceptelor de bază* în arbori disjuncți,
arborii având legături între ei via definiții și restricții

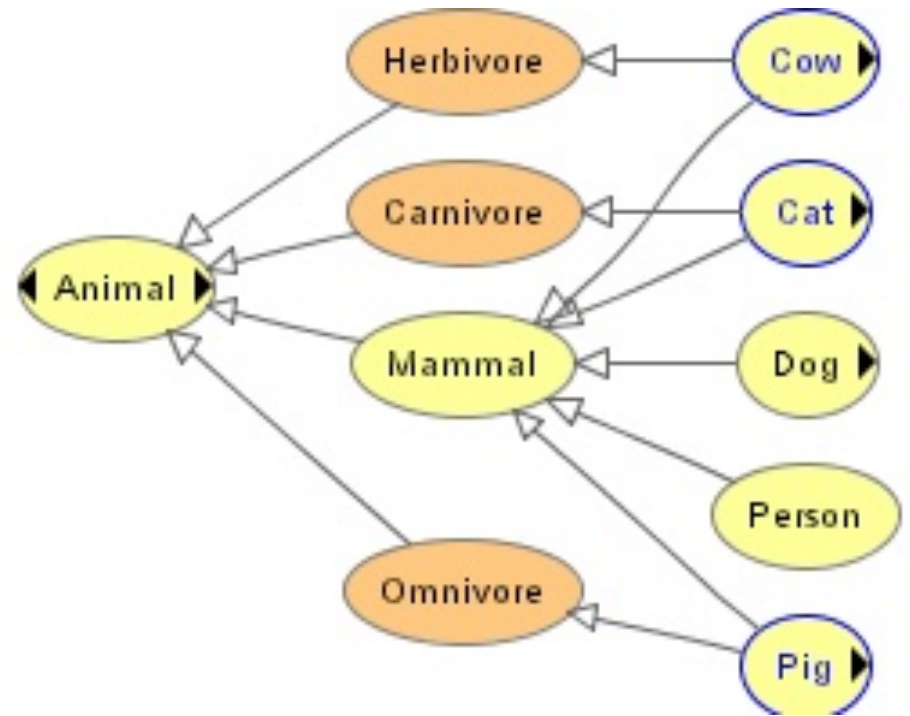
aspecte importante

Exemplu: o ontologie privitoare la animale

înainte de clasificare
arbore de concepte



după clasificare
digraf aciclic



Dezvoltarea unei **ontologii normalizate** presupune:

1. **Identificarea conceptelor de bază de sine-stătătoare**
(cu comentarea celor mai puțin evidente)
2. **Separarea conceptelor în arbori de concepte**
(poate implica specificarea unor proprietăți/
concepte auxiliare)
3. **Identificarea relațiilor**
(comentarea acelor relații care nu sunt evidente)
4. **Crearea descrierilor și definițiilor**
(cu oferirea de parafraze pentru fiecare)
5. **Identificarea modului în care entitățile-cheie
trebuie clasificate** (crearea unor teste de regresie)
6. **Utilizarea clasificatorului pentru a genera digraful**
7. **Verificarea satisfacerii testelor**

considerații privind modelarea

„Trusa” ontologistului

cerințe

formalisme (construcții logice)

ontologii existente

resurse de cunoștințe informale

convenții și practici

instrumente

adaptare după Aldo Gangemi (2008)

considerații privind modelarea

„Trusa” ontologistului

cerințe – *e.g.*, „un model ideal de specificare a lumii” 😊

construcții logice: *subClassOf*, restricții de cardinalitate,...

ontologii existente: DOLCE, FOAF, schema.org, multe altele

resurse de cunoștințe informale: servicii Web, API-uri,...

convenții și practici – inclusiv șabloane de proiectare

instrumente – editoare, *reasoners*, convertoare, sisteme de stocare, GPT-uri etc. (Apache Jena, ELK, Openllet, OWL API, Protégé, RDFLib, Stardog, Triplr, Virtuoso, VocBench,...)

considerații privind modelarea

Înțelegerea clară a distincțiilor (Franconi, 2003)

concept (clasă) *vs.* individ

concept (clasă) *vs.* proprietate (rol)

entitate definită *vs.* entitate dedusă

considerații privind modelarea

Clasă (concept) *versus* instanță (individ)

o mulțime de indivizi este una numărabilă, discretă

spațiul privitor la concepte e ideal continuu și infinit

considerații privind modelarea

Clasă (concept) *versus* instanță (individ)

fiecare individ posedă o identitate clară

considerații privind modelarea

Clasă (concept) *versus* instanță (individ)

dacă două concepte au descrieri echivalente,
atunci desemnează același concept

descrierile privitoare la indivizi
pot fi alterate (pot evolua)

considerații privind modelarea

Clasă (concept) *versus* instanță (individ)

uzual, actualizările vizând indivizii
nu schimbă ierarhia conceptelor

considerații privind modelarea

Clasă (concept) *versus* instanță (individ)

alegerea „clasă vs. individ” depinde de granularitate

uzual, numele de clase și indivizi
sunt desemnate de substantive

considerații privind modelarea

Clasă (concept) *versus* proprietate (rol)

nu e ușor de decis dacă o entitate trebuie să fie modelată
ca un concept sau drept proprietate

considerații privind modelarea

Clasă (concept) *versus* proprietate (rol)

nu e ușor de decis dacă o entitate trebuie să fie modelată
ca un concept sau drept proprietate

Persoana este un concept (clasă)
Mama ca și concept nu există dacă nu considerăm
ce rol joacă într-o relație parentală
(este un concept dependent de o relație)

considerații privind modelarea

Clasă (concept) *versus* proprietate (rol)

nu e ușor de decis dacă o entitate trebuie să fie modelată
ca un concept sau drept proprietate

convenție:

alegerea de nume neambigue pentru proprietăți
folosirea verbelor – *hasParent, hasGender* etc.

versus

recurgerea la substantive – *age, affiliation, alumniOf,...*

considerații privind modelarea

Fiecărui concept ales i se pot asocia:

proprietăți intrinseci
caracteristice naturii conceptului

proprietăți extrinseci
externe – pot să se modifice pe parcurs

considerații privind modelarea

Fiecărui concept ales i se pot asocia:

proprietăți intrinseci

caracteristice naturii conceptului
exemplu: identitatea unei persoane

proprietăți extrinseci

externe – pot să se modifice pe parcurs
e.g., ocupația unei persoane: student, designer, artist

considerații privind modelarea

Fiecărui concept ales i se pot asocia:

părți, fragmente – în cazul obiectelor structurate/compuse

fizice

abstracte

considerații privind modelarea

Fiecărui concept ales i se pot asocia:

părți, fragmente – în cazul obiectelor structurate/compuse

fizice

„studentii dintr-un an”, „componentele unei tablete”,
„boabele unui strugure” etc.

abstracte

„prezentările unei materii”, „subiectele unui *blog*”,...

considerații privind modelarea

Raționamentele automate privitoare la indivizi
sunt dificil de realizat

ontologiile vizează în special clasele
ontologii $\neq$ baze de date

considerații privind modelarea

O ontologie urmează un ciclu de viață (*lifecycle*),
fiind creată, evaluată, ajustată, exploatată
ca și alt artefact software

considerații privind modelarea

O ontologie urmează un ciclu de viață (*lifecycle*),
fiind creată, evaluată, ajustată, exploatată
ca și alt artefact software

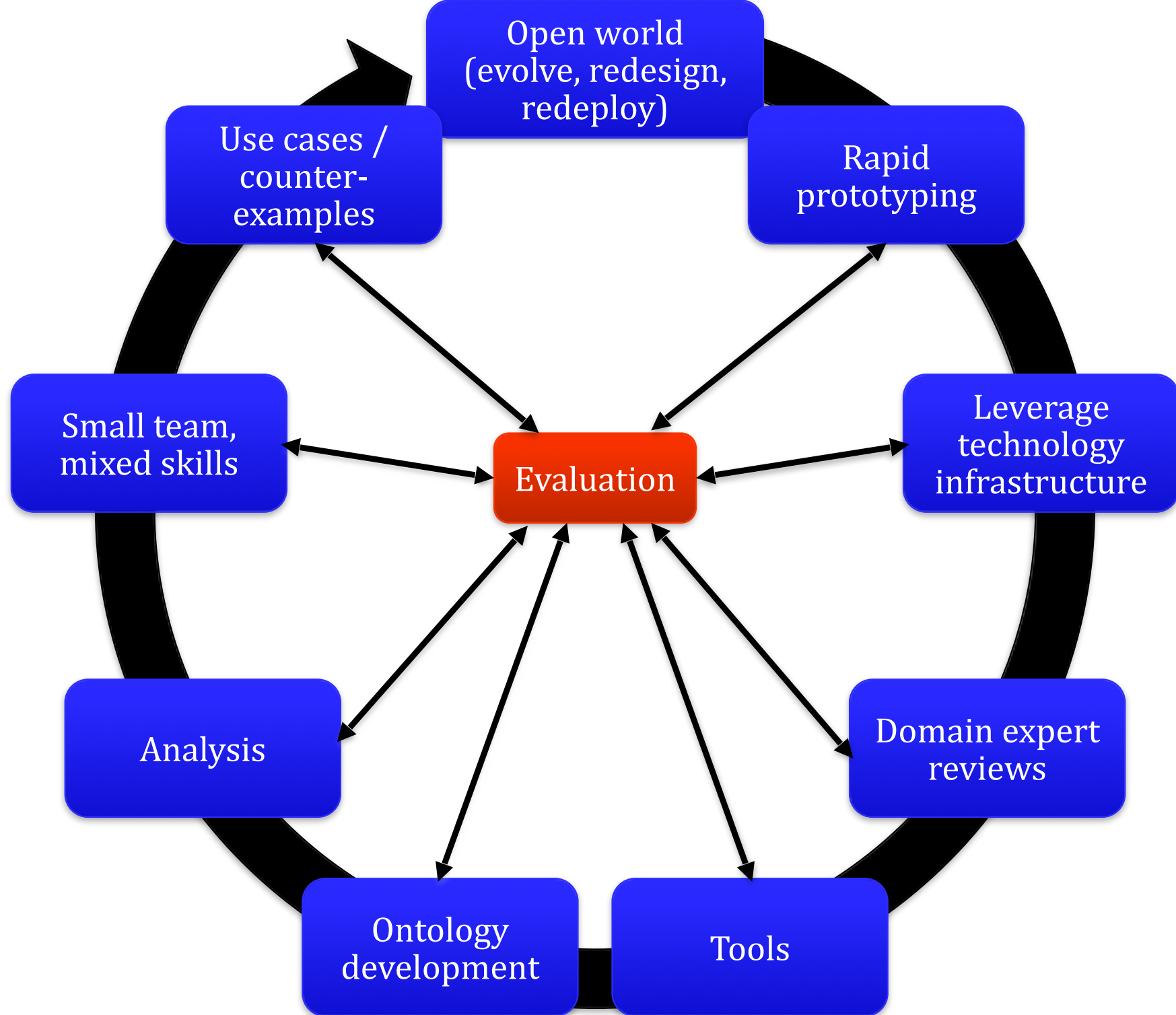
aspecte de interes:

datele

tipurile de *workflow*-uri

șabloanele de proiectare (reguli de bună practică)

utilizarea propriu-zisă



adaptare după J. Benedict, D. McGuinness, P. Fox (2007)

considerații privind modelarea

Verificarea manierei de modelare

un catalog al greșelilor comune (*pitfalls*)

María Poveda-Villalón, Asunción Gómez-Pérez, Mari Carmen Suárez-Figueroa, “OOPS!: A Pitfall-Based System for Ontology Diagnosis”, *Innovations, Developments, and Applications of Semantic Web and Information Systems*, IGI Global, 2018

Evaluation results

It is obvious that not all the pitfalls are equally important; their impact in the ontology will depend on multiple factors. For this reason, each pitfall has an importance level attached indicating how important it is. We have identified three levels:

- **Critical** 🛑 : It is crucial to correct the pitfall. Otherwise, it could affect the ontology consistency, reasoning, applicability, etc.
- **Important** 🟡 : Though not critical for ontology function, it is important to correct this type of pitfall.
- **Minor** 🟢 : It is not really a problem, but by correcting it we will make the ontology nicer.

[\[Expand All\]](#) | [\[Collapse All\]](#)

Results for P04: Creating unconnected ontology elements.	2 cases Minor 🟢
Results for P08: Missing annotations.	2 cases Minor 🟢
Results for P11: Missing domain or range in properties.	7 cases Important 🟡
Results for P12: Equivalent properties not explicitly declared.	2 cases Important 🟡
Results for P13: Inverse relationships not explicitly declared.	27 cases Minor 🟢
Results for P22: Using different naming conventions in the ontology.	ontology* Minor 🟢
Results for P34: Untyped class.	4 cases Important 🟡
Results for P35: Untyped property.	1 case Important 🟡
Results for P36: URI contains file extension.	ontology* Minor 🟢
Results for P41: No license declared.	ontology* Important 🟡
The ontology metadata omits information about the license that applies to the ontology.	
*This pitfall applies to the ontology in general instead of specific elements.	
SUGGESTION: symmetric or transitive object properties.	6 cases

Verificarea manierei de modelare caz concret: Wikidata

Serghei Stratan, *Taxonomy browsing and ontology evaluation for Wikidata* (2016)

www.logic.at/emcl2016/Serge%20EMCL_Workshop_Presentation.pdf

Criteria for the Wikidata ontology evaluation:

- 1 Cycles detection
- 2 Self-loops detection
- 3 Root classes
- 4 Finding classes which have more than 100 direct subclasses
- 5 Errors of relation properties
- 6 Redundancies of relation properties

exemplificare: OnToology

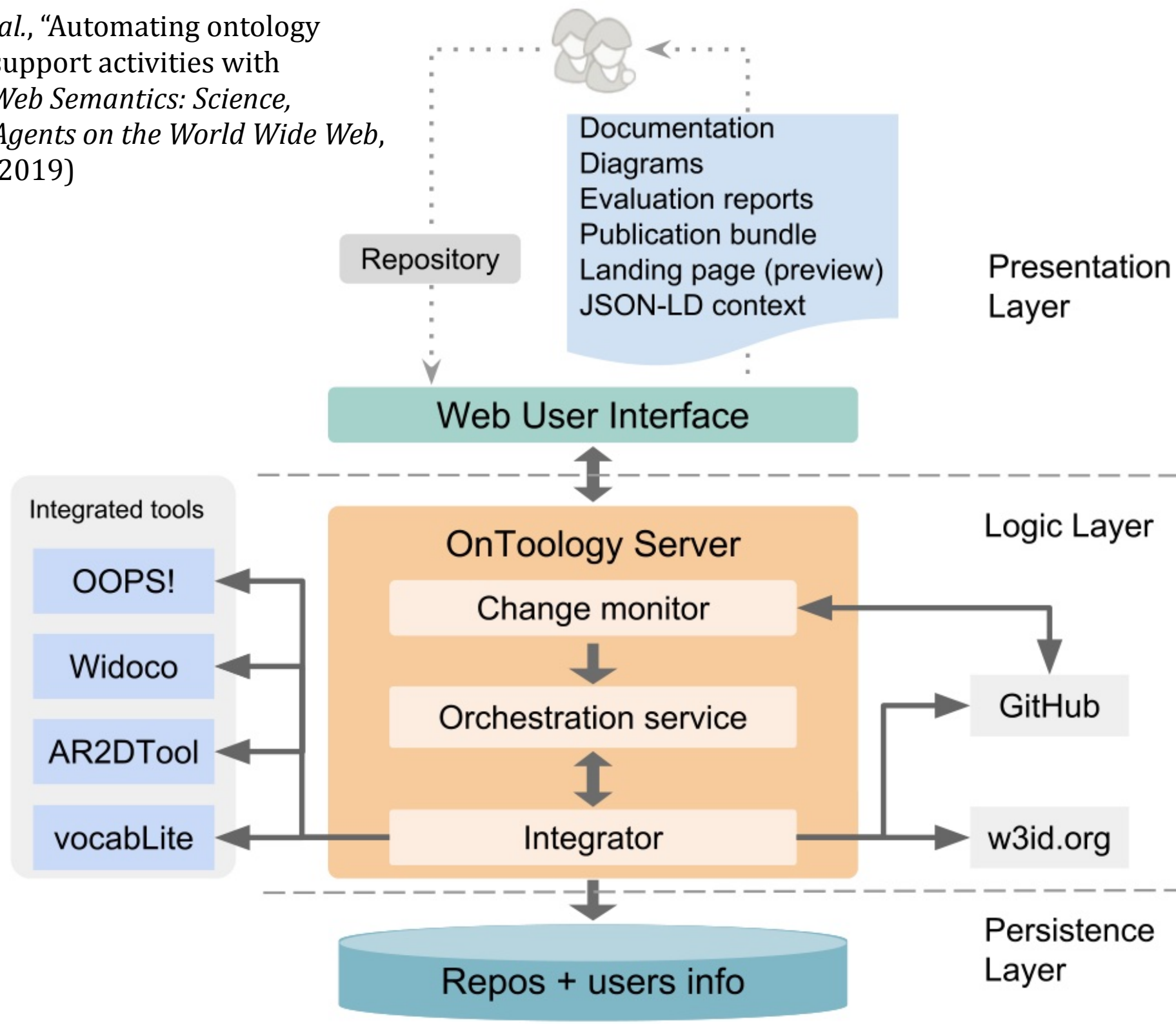
Automatizarea proceselor vizând ingineria ontologică
(managementul activităților de documentare, evaluare,
publicare și control al versiunilor pe baza Git)

sistem Web implementat în Python

integrează diverse instrumente deschise

github.com/ontology

A. Alobaid *et al.*, “Automating ontology engineering support activities with OnToolology”, *Web Semantics: Science, Services and Agents on the World Wide Web*, 57, Elsevier (2019)



studiu de caz



Modelarea unei ontologii privitoare la animale



studiu de caz

Intenții și scopuri

realizarea unei cărți electronice (*e-book*)
despre animale, oferind informații precum:
 locul unde trăiesc
 ce consumă – carnivore, ierbivore, omnivore
 cât de periculoase pot fi
ce anatomie de bază au – *e.g.*, numărul membrelor,...
 etc.

adaptare după Alan Rector – www.cs.man.ac.uk/~rector/

studiu de caz

Colectarea conceptelor

se pot folosi carduri de concepte

se lucrează în grupuri mici de persoane

se pot atrage experți ai domeniului

studiu de caz

Dog

Cat

Cow

Person

Tree

Grass

Herbivore

Male

Female

Carnivore

Plant

Animal

Fur

Child

Parent

Mother

Father

Dangerous

Pet

Domestic animal

Farm animal

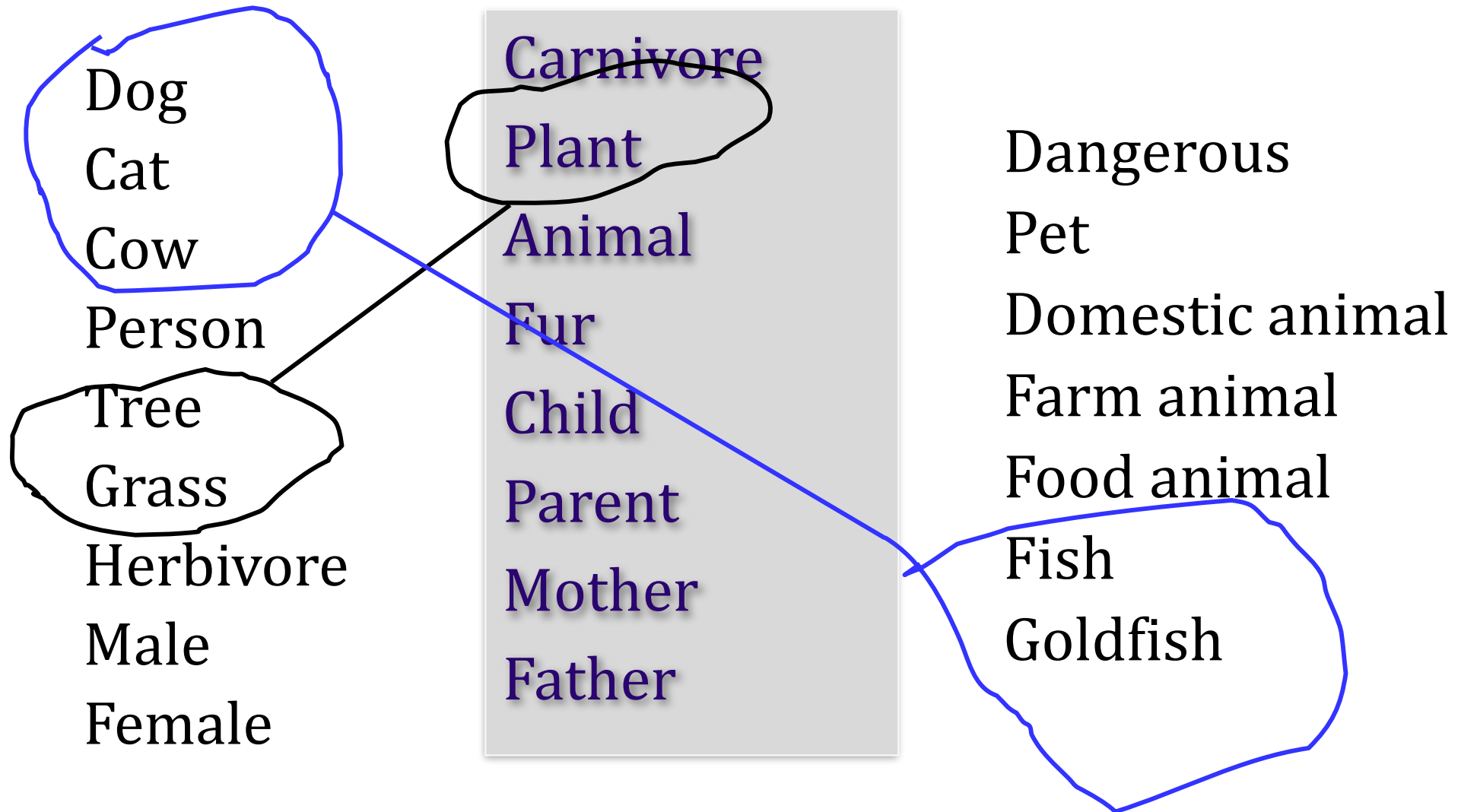
Food animal

Fish

Goldfish

clase posibile

studiu de caz: organizarea conceptelor



studiu de caz

Extinderea conceptelor

gruparea unor concepte
și determinarea caracteristicilor comune:

Plant, Animal $\mapsto$ *Living Thing*

ar putea fi adăugați ulterior și termeni ca *Bacteria, Fungi*

Cat, Dog, Cow, Person $\mapsto$ *Mammal*

alți candidați ar putea fi *Goat, Sheep, Horse, Lion*
etc.

studiu de caz

Pentru facilitarea clasificării, colecțiile de subclase trebuie declarate explicit disjuncte

exemplificare:

Plant disjointWith Animal

studiu de caz

Alegerea unor axe conceptuale

adăugarea de termeni abstracți

de exemplu: “*Living thing*”

studiu de caz

Alegerea unor axe conceptuale

identificarea relațiilor

de pildă: “*eats*”, “*owns*”, “*parent of*”

studiu de caz

Alegerea unor axe conceptuale

identificarea entităților ce pot fi definite
(în relație cu altele)

e.g., “child”, “parent”, “Mother”, “Father”

studiu de caz

Alegerea unor axe conceptuale

numirea explicită a conceptelor

utilizarea cunoștințelor anterioare
(*background knowledge*)



axiome

Living Thing

Animal

Mammal

Cat

Dog

Cow

Person

Fish

Goldfish

Plant

Tree

Grass

Fruit

Modifiers

Domestic

Pet

Farmed

Food

Wild

Health

healthy

sick

Gender

Male

Female

Age

Adult

Child

Relations

eats

owns

parent-of

...

Definables

Carnivore

Herbivore

Child

Parent

Mother

Father

Food Animal

studiu de caz

Stabilirea entităților de sine-stătătoare

„lucruri” ce există conform substantivelor lor
oameni, animale, edificii, acțiuni, procese,...
se utilizează majoritatea substantivelor

modificatori (*modifiers*)

„lucruri” ce se pot preschimba în altele
se folosesc adjectivele și adverbele

studiu de caz

Reorganizăm conceptele, grupând entitățile care pot fi definite în arbori separați

- constituirea conceptelor de bază (*primitives*)

Self_standing

Living Thing

Animal

Mammal

Cat

Dog

Cow

Person

Fish

Gold

fish

Plant

Tree

Grass

Fruit

Modifiers

Domestication

Domestic

Wild

Use

Food

Pet

Risk

Dangerous

Safe

Gender

Male

Female

Age

Adult

Child

Relations

eats

owns

parent-of

...

Definables

Carnivore

Herbivore

Child

Parent

Mother

Father

Food Animal

Self_standing

Living Thing ←

Animal

Mammal

Cat

Dog

Cow

Person

Fish

Gold
fish

Plant

Tree

Grass

Fruit

Concept abstract incluzând toate entitățile vii
– aici, restricționat la plante și animale.

adăugarea comentariilor
clarificatoare (parafraze)

studiu de caz

Identificarea constrângerilor privitoare la domeniu (*domain*) și interval (*range*) pentru fiecare proprietate:

Animal *eats* Living_thing

<i>eats</i>	domain:	Animal
	range:	Living_thing

Person *owns* Living_thing except Person

<i>owns</i>	domain:	Person
	range:	Living_thing and not Person

Living_thing *parent_of* Living_thing

<i>parent_of</i>	domain:	Animal
	range:	Animal

studiu de caz

Pentru entitățile ce pot fi definite,
trebuie comentate și formalizate
definițiile acestora în termeni de **primitive**,
relații și alte entități ce pot fi specificate

studiu de caz

Exemplu

“A ‘Parent’ is an animal that is the parent of some other animal” – ignore plants for now

Parent = Animal and *parent_of* **some** Animal

studiu de caz

Exemplu

“A ‘Herbivore’ is an animal that eats only plants”

NB: All animals eat some living thing

Herbivore = Animal and *eats* **only** Plant

studiu de caz

Stabilirea proprietăților ce pot apărea
la nivel de clasă

studiu de caz

Ce se poate afirma privitor la *toți* membrii clasei?

exemplificare: *eats*

all cows eat some plants

all cats eat some animals

all pigs eat some animals and eat some plants

studiu de caz

Verificarea clasificării

“Cows should be Herbivores”

cows are animals and, amongst other things,

eat some grass and

eat some leafy_plants

studiu de caz

Verificarea clasificării

trebuie specificată **axioma de închidere** (***closure axiom***)

cows are animals and, amongst other things,

*eat some plants and
eat only plants*

studiu de caz

Examinarea listei de modifcatori

identificarea modifcatorilor
care au valori mutual exclusive
Domestication, Risk, Gender, Age

precizarea mai precisă a unora
Age $\mapsto$ Age_group

Modifiers

Domestication

Domestic

Wild

Feral

Risk

Dangerous

Risky

Safe

Gender

Male

Female

Age

Infant

Toddler

Child

Adult

Elderly

noua listă de modificatori

studiu de caz

Specificarea valorilor (soluția 1)

partiții de valori: clase ce partiționează o calitate

Dangerousness cu sub-calitățile disjuncte

Dangerous, Risky, Safe

studiu de caz

Specificarea valorilor (soluția 1)

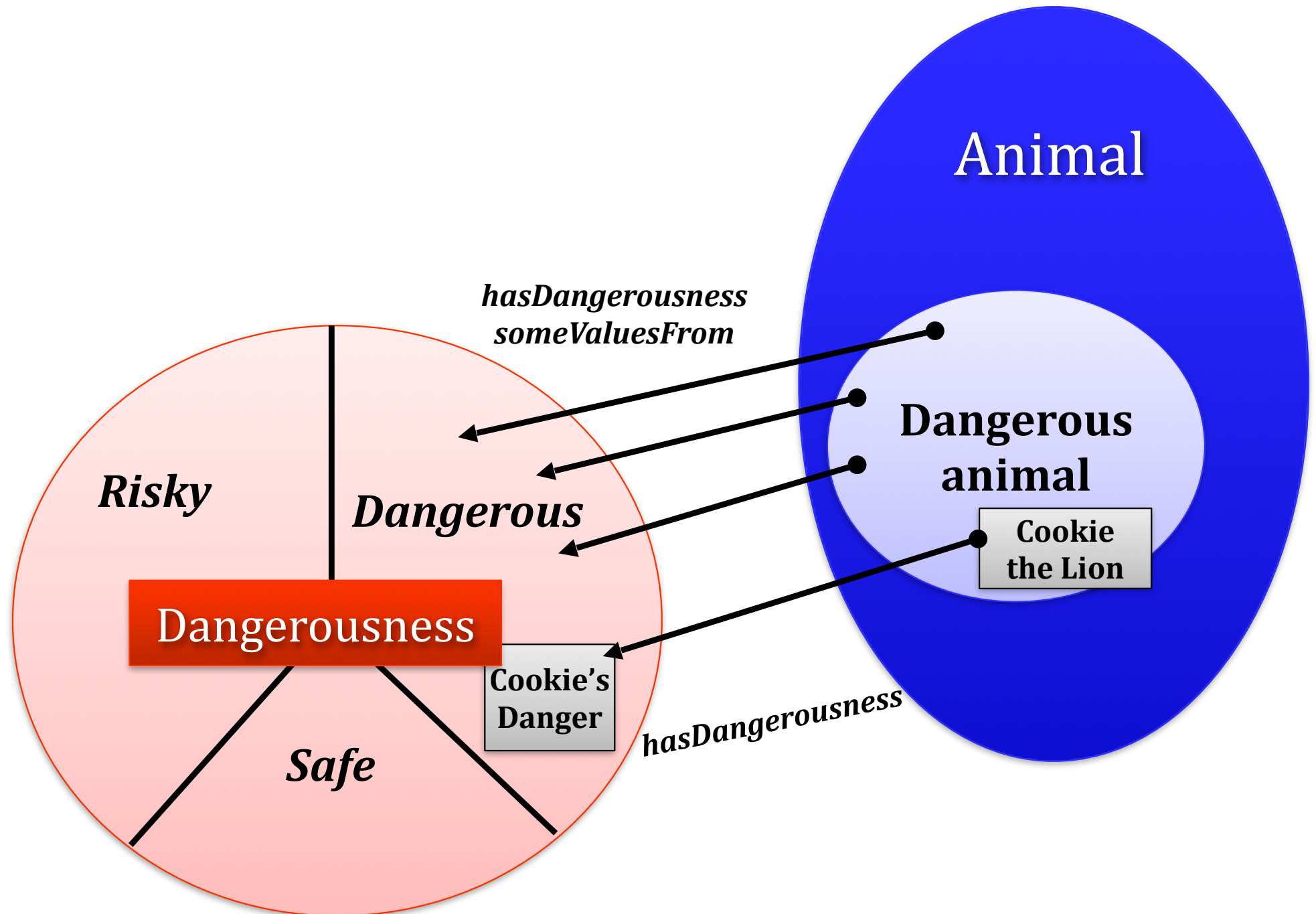
partiții de valori: clase ce partiționează o calitate

se poate specifica proprietatea funcțională

hasDangerousness : **Animal** $\mapsto$ **Dangereousness**

Dangerous_animal = **Animal** and
hasDangerousness *some* Dangerous

diagrama partițiilor de valori



studiu de caz

Specificarea valorilor (soluția 1)
partiții de valori: clase ce partiționează o calitate

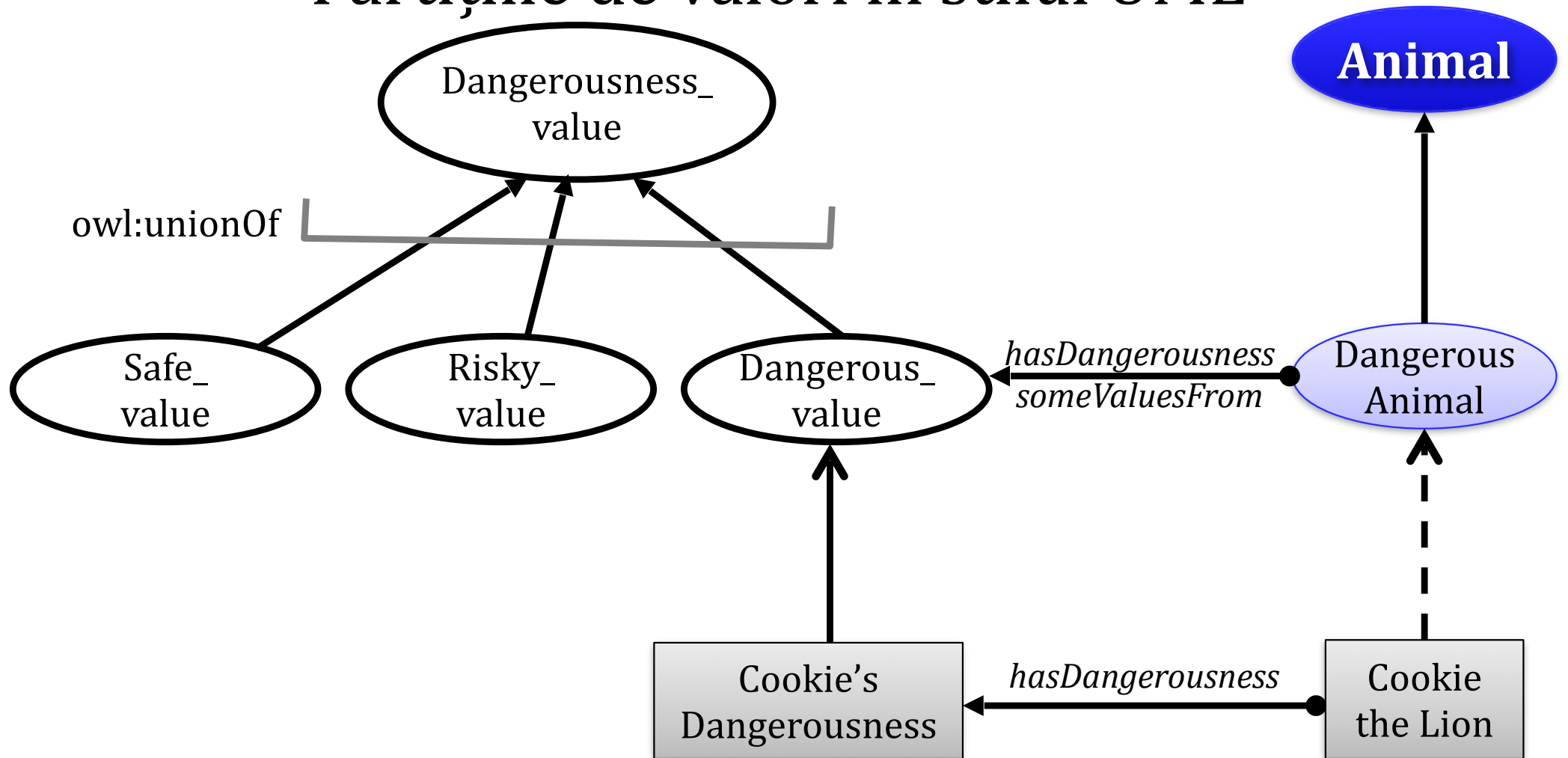
calitatea



partițiile
de valori

studiu de caz

Partițiile de valori în stilul UML



studiu de caz

Specificarea valorilor (soluția 2)

valori simbolice: indivizii pot enumera toate valorile unei calități (caracteristici)

exemplu: calitatea specificată este *Gender_value*, pentru fiecare valoare existând o mulțime de indivizi

valorile vor fi diferite, date ca o enumerare:

Gender_value = { *male*, *female* }

studiu de caz

Specificarea valorilor (soluția 2)

valori simbolice: indivizii pot enumera toate valorile unei calități (caracteristici)

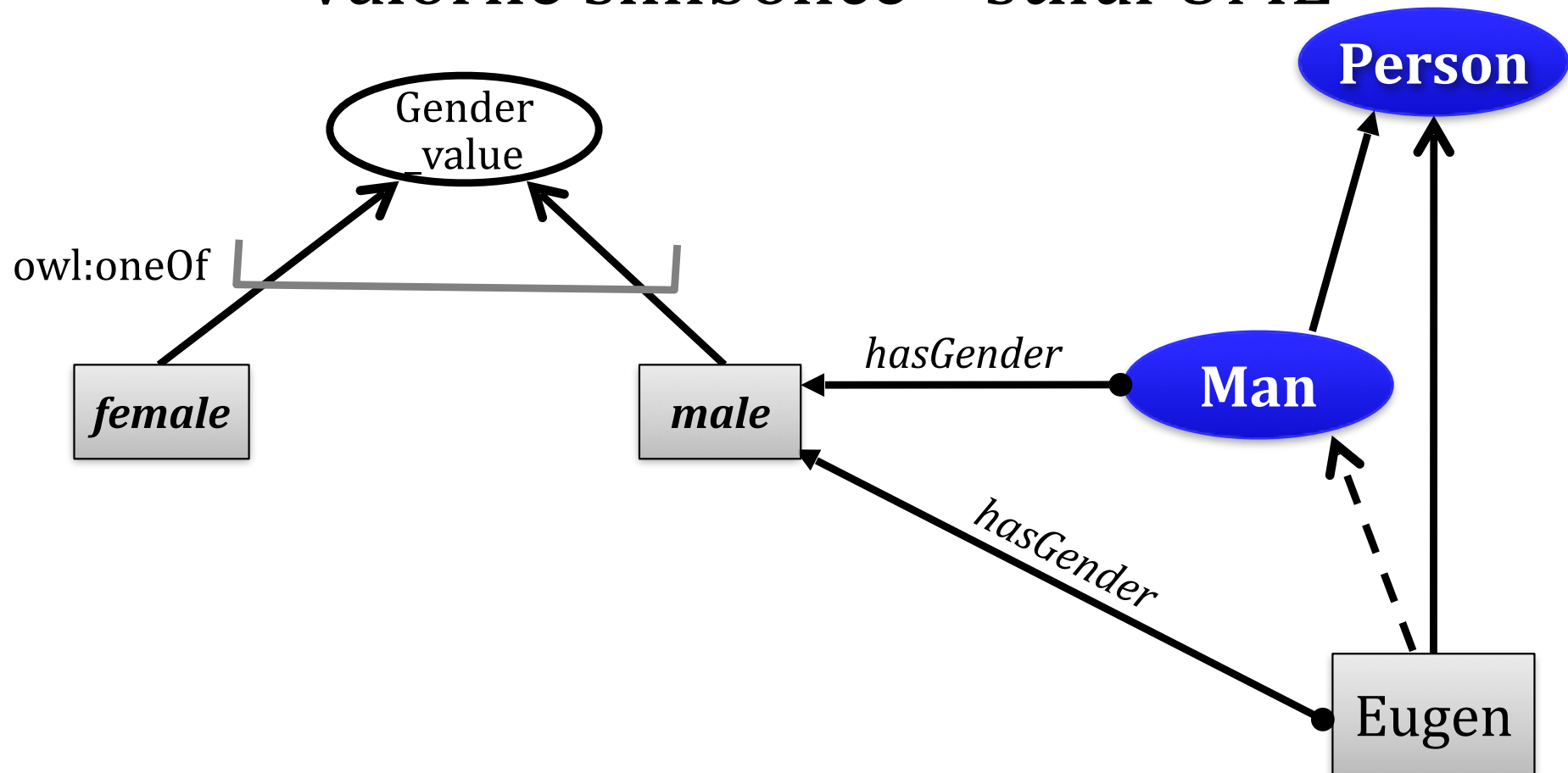
se definește o proprietate funcțională

***hasGender* : Animal $\mapsto$ Gender_value**

Male_animal = Animal and ***hasGender*** is *male*

studiu de caz

Valorile simbolice – stilul UML



studiu de caz

Specificarea valorilor (*design pattern*)

valori distincte exprimate via *Disjoint* pentru clase
sau *allDifferent* pentru indivizi

studiu de caz

Specificarea valorilor (*design pattern*)

valorile „acoperă” un tip de date

Calitate = Part_1 *or* Part_2 *or* Part_3 *or* ... *or* Part_n

sau

Calitate = $\{ v_1 , v_2 , v_3 , ..., v_n \}$

proprietățile exprimate sunt deseori funcționale

studiu de caz

Exprimarea rolurilor (proprietăților)

apare necesitatea distingerei rolurilor
pe care le joacă entitățile în diferite situații

studiu de caz

Exprimarea rolurilor (proprietăților)

apare necesitatea distingerei rolurilor
pe care le joacă entitățile în diferite situații

exemple:

“pet” versus “farm animal”

„elev” *versus* „student” *versus* „profesor”

„medic” *versus* „pacient”

studiu de caz

Exprimarea rolurilor (proprietăților)

deseori, calificările trebuie distinse față de roluri

e.g., o persoană poate avea calificare de medic,
dar joacă rol de pacient

studiu de caz

Exprimarea rolurilor (proprietăților)
uzual, rolurile desemnează relații

Animal
Food_animal
Cow
Horse
Pet_animal
Horse
Dog

Animal
Mammal
Cow
Horse
Dog
Animal_use_role
Food_role
Pet_role

Food_animal =
Animal and
has_role
some Food_role

Pet_animal =
Animal and
has_role
some Pet_role

studiu de caz

Exprimarea rolurilor (proprietăților)

situație:

Horse plays_role some Food_role

► toți caii au rol de animale bune de consum

o afirmație prea categorică

studiu de caz

Exprimarea rolurilor (proprietăților)

soluții:

ignorarea problemei într-un anumit context

sau

înlocuirea lui *has_role* cu *may_have_role*

se poate specifica

Food_Horse = Horse and **has_role** Food_role

studiu de caz

Observație:

anumite entități pot fi deduse pe baza inferențelor
– via *reasoners* (e.g., FaCT++, HermiT, Openllet)

în cazul de față:

Grass and *Leafy_plants* are both kinds of *Plant*



revezi exemplele
din cursul anterior

studiu de caz

Comentarea și parafrizarea tuturor claselor
din cadrul ontologiei

foarte util pentru alte persoane care vor folosi
și/sau vor extinde ontologia

studiu de caz

Oferirea unui set de clase de probă și
a unui cadru de testare a ontologiei

clase de probă = clase suplimentare care pot să fie
sau nu satisfiabile sau clasificabile
într-un context particular – „excepțiile de la regulă”

în ontologia finală, vor fi eliminate

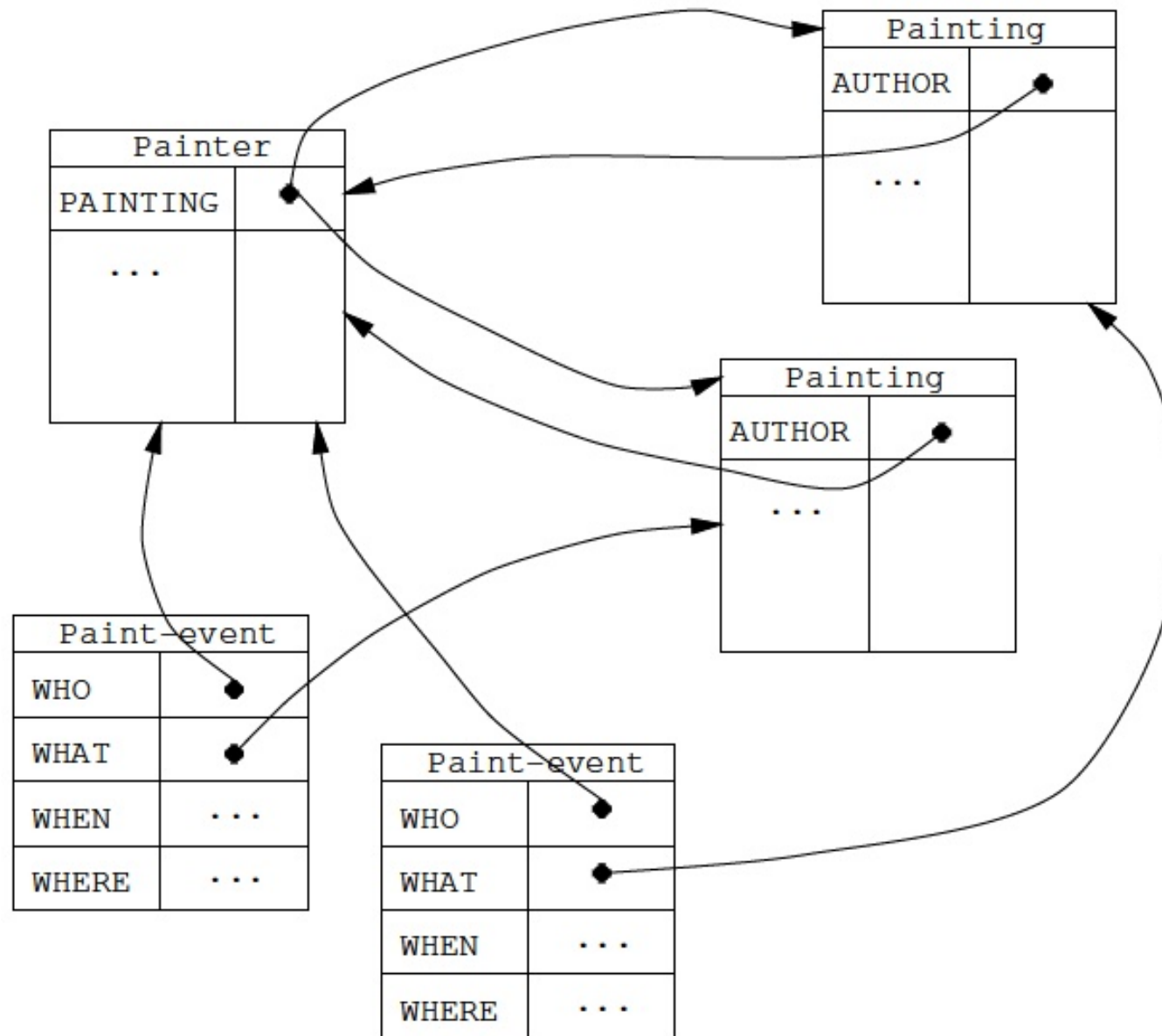
studiu de caz

Modelarea unui *web* privitor la un muzeu de artă



eronată!

adaptare după Enrico Franconi



TBox redundant + ciclic

► nu se pot detecta ușor inconsistențele

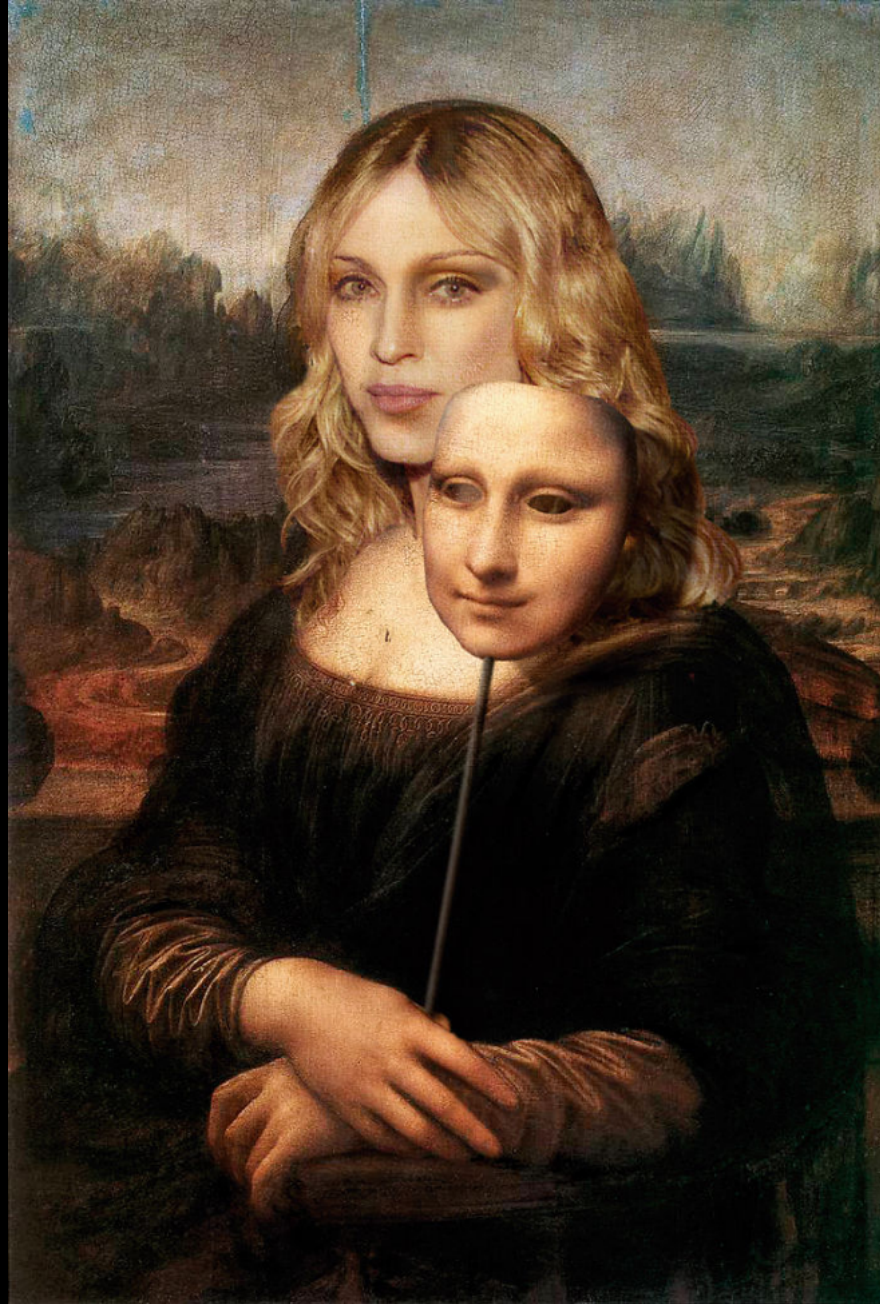
studiu de caz

La nivel de ABox se specifică:

Dali este pictor, *Pescuit de ton* este tablou,
tabloul *Pescuit de ton* al lui Dali se găsește în Madrid
tabloul *Pescuit de ton*, pictat de Dali în 1966,
se găsește în Madrid etc.

date specificate redundant:

Painter (dali), **PAINTING** (dali, tuna-fishing),
Painting (tuna-fishing), **AUTHOR** (tuna-fishing, dali),...



În cazul modelării cunoștințelor,
s-ar putea recurge la șabloane de proiectare?

șabloane de proiectare

Clasificare pe baza seriei de evenimente WOP

Workshop on Ontology Patterns

ontologydesignpatterns.org • [odpa.github.io](https://github.com/ontologydesignpatterns/odpa)

la nivel logic (*logical*)

arhitecturale (*architectural*)

lexico-sintactice (*lexico-syntactic*)

referitoare la conținut

re-engineering

Taxonomia șabloanelor de proiectare:

Structural

Logical

Architectural

Correspondence

Re-engineering

Alignment

Content

Reasoning

Presentation

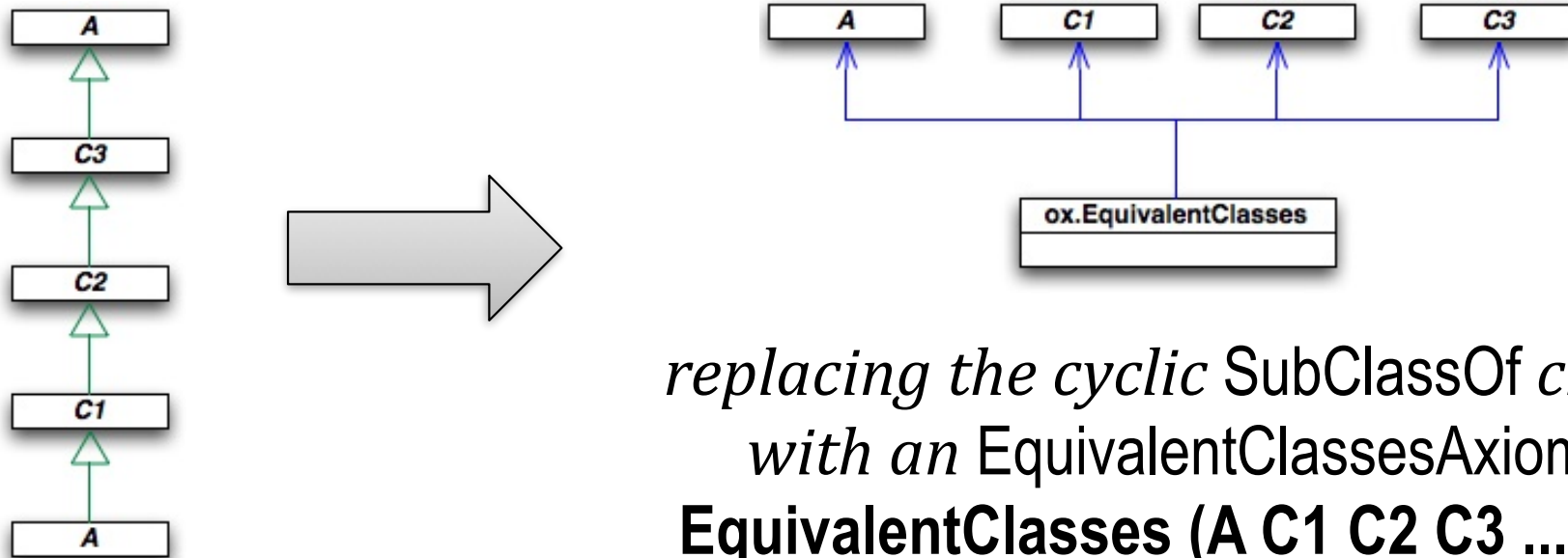
Naming

Annotation

Lexico-Syntactic

detalii la ontologydesignpatterns.org/wiki/OPTypes

șabloane de proiectare



*replacing the cyclic SubClassOf chain
with an EquivalentClassesAxiom:*
EquivalentClasses (A C1 C2 C3 ... Cn)

CyclingSubClassOf – șablon de tip *re-engineering*
ontologydesignpatterns.org/wiki/Submissions:CyclicSubClassOf

șabloane de proiectare

Pot fi folosite și în cadrul alinierii de ontologii

exemple:

Attribute Transformation

(String Operation, Value Conversion, Data Operation),

Class to Attribute, Class Union,

Equivalent Relation, Relation Negation, Inverse Relation

etc.

șabloane de proiectare

Specifice unor domenii de interes

exemplificări:

pentru *business*, se poate recurge la șabloanele
Action, CommunicationEvent, Time indexed person role

în cazul fluxurilor de lucru (*workflow-uri*), se pot folosi
Reaction, Sequence, TaskExecution, Transition

pescuit (*fishery*): *GearSpecies, GearVessel, GearWaterArea*

șabloane de proiectare: studii de caz

M. Atzeni, M. Atzori, *CodeOntology: RDF-ization of Source Code*,
ISWC 2017

videolectures.net/iswc2017_atzeni_source_code/

N. Petersen *et al.*, *Realizing an RDF-based Information Model
for a Manufacturing Company*, ISWC 2017

iswc2017.ai.wu.ac.at/wp-content/uploads/papers/MainProceedings/426.pdf

M. Glockner, A. Ludwig, *LoSe ODP – An Ontology Design Pattern
for Logistics Services*, WOP 2016

ontologydesignpatterns.org/wiki/images/f/fb/WOP2016_paper_14.pdf

V. R. Doncel, *RDFChess*, 2019

github.com/vroddon/RDFChess



Cum integrăm diferite ontologii
descriind aceleași cunoștințe,
dar prezentând definiții,
puncte de vedere și noțiuni
diferite?

realitate

existența unor ontologii diverse,
modelând puncte de vedere diferite



“Basically, we’re all trying to say the same thing.”

© The New Yorker collection. All rights reserved.
From The New Yorker Book of Technology Cartoons.

“Basically, we’re all trying to say the same thing.”

preliminarii

Expresivitatea limbajului ontologic folosit
versus
complexitatea inferențelor ce pot fi efectuate

preliminarii

Aspecte de interes

(Calvanese, Giacomo & Rodriguez-Muro, 2008):

utilizarea construcțiilor ontologice oferite de OWL

► efectuarea de **inferențe**

preliminarii

Aspecte de interes

(Calvanese, Giacomo & Rodriguez-Muro, 2008):

recurgerea la volum mare de date (indivizi)

- ▶ (re)utilizare a unor baze de date

preliminarii

Aspecte de interes

(Calvanese, Giacomo & Rodriguez-Muro, 2008):

flexibilitate privind realizarea interogărilor

► limbaje de interogare – specifice (SPARQL)

preliminarii

Aspecte de interes

(Calvanese, Giacomo & Rodriguez-Muro, 2008):

realizarea de asocieri între date,
păstrând sursele originare

- asocieri (*mappings*) – e.g., baze de date ↔ *triplestores*

preliminarii

Context: integrarea datelor – *data integration*

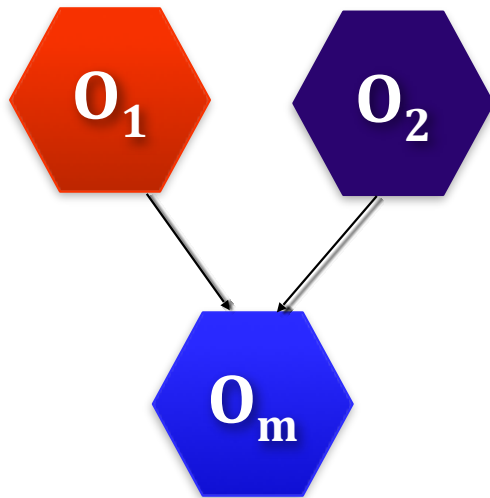
data integration is the problem of providing unified and transparent access to a set of autonomous and heterogeneous sources

Bernstein & Haas, *Communications of the ACM*, 2008

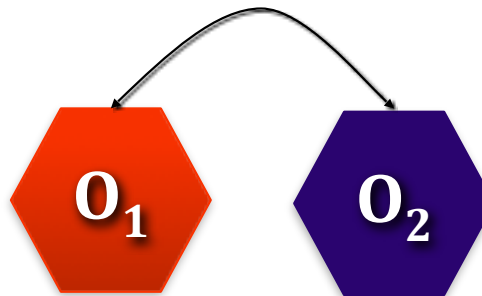
necesități

Crearea corespondențelor dintre ontologii

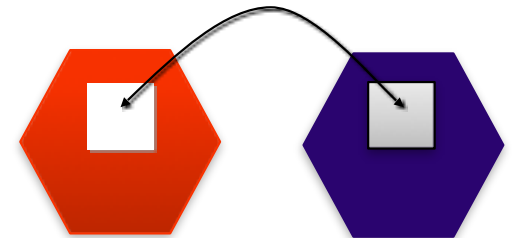
merging



mapping



articulation



potrivirea ontologiilor

Context:
integrarea datelor

scheme de baze de date (relaționale)
specificații provenite din inteligența artificială
Web (semantic) – RDF, JSON(-LD), OWL,...

potrivirea ontologiilor

Motive ale existenței nepotrivirilor – *mismatches*
conform (Noy, 2005)

reprezentarea subiectivă a unui domeniu (lume, *web*)

exemplu:

carte pentru copii despre animale vs. compendiu zoologic

potrivirea ontologiilor

Motive ale existenței nepotrivirilor – *mismatches*
conform (Noy, 2005)

existența diferitelor cerințe și *workflow*-uri
privitoare la (tipuri de) aplicații software

exemplificări:

e-business vs. e-learning vs. e-entertainment,
maniere diverse de interacțiune cu utilizatorul

potrivirea ontologiilor

Motive ale existenței nepotrivirilor – *mismatches*
conform (Noy, 2005)

folosirea altor convenții de reprezentare a cunoștințelor

potrivirea ontologiilor

Tipuri de nepotriviri
la nivel...

de limbaj

conceptual (ontologic)

potrivirea ontologiilor

Nepotriviri la nivel de limbaj

diferențe de sintaxă, expresivitate sau semantică
a limbajelor ontologice actuale

potrivirea ontologiilor

Nepotriviri la nivel ontologic

diferențe vizând structura semantică

exemple: termeni diferiți definind același concept,
același termen descriind mai multe concepte, convenții/
paradigme de modelare eterogene,
puncte de vedere diferite etc.

potrivirea ontologiilor – exemplu

www.aktors.org/ontology/ vs. ebiquity.umbc.edu/ontology/

scop:

modelare de cunoștințe din domeniul academic
periodice, publicații compuse vs. absolvenți, invitați

potrivirea ontologiilor – exemplu

www.aktors.org/ontology/ vs. ebiquity.umbc.edu/ontology/

nume diferite pentru același concept:

PhD-Student *versus* **PhDStudent**

potrivirea ontologiilor – exemplu

www.aktors.org/ontology/ vs. ebiquity.umbc.edu/ontology/

specificare diferită a conceptelor:

Publication – restricții privitoare la citări, referințe biblio.

versus

Publication – restricții referitoare la rezumat

potrivirea ontologiilor – exemplu

www.actors.org/ontology/ vs. ebiquity.umbc.edu/ontology/

convenții de modelare diferite:

Journal (clasă) vs. **journal** (proprietate)

potrivirea ontologiilor – exemplu

www.aktors.org/ontology/ vs. ebiquity.umbc.edu/ontology/

granularitate:

Professor-In-Academia vs. **adjunct, affiliated, associate,...**

potrivirea ontologiilor – exemplu

www.aktors.org/ontology/ vs. ebiquity.umbc.edu/ontology/

convenții de modelare și nivel de detaliere:

proprietatea **address** compusă
din alte fragmente de informație

versus

proprietatea **address** de tip șir de caractere (nestructurat)

potrivirea ontologiilor

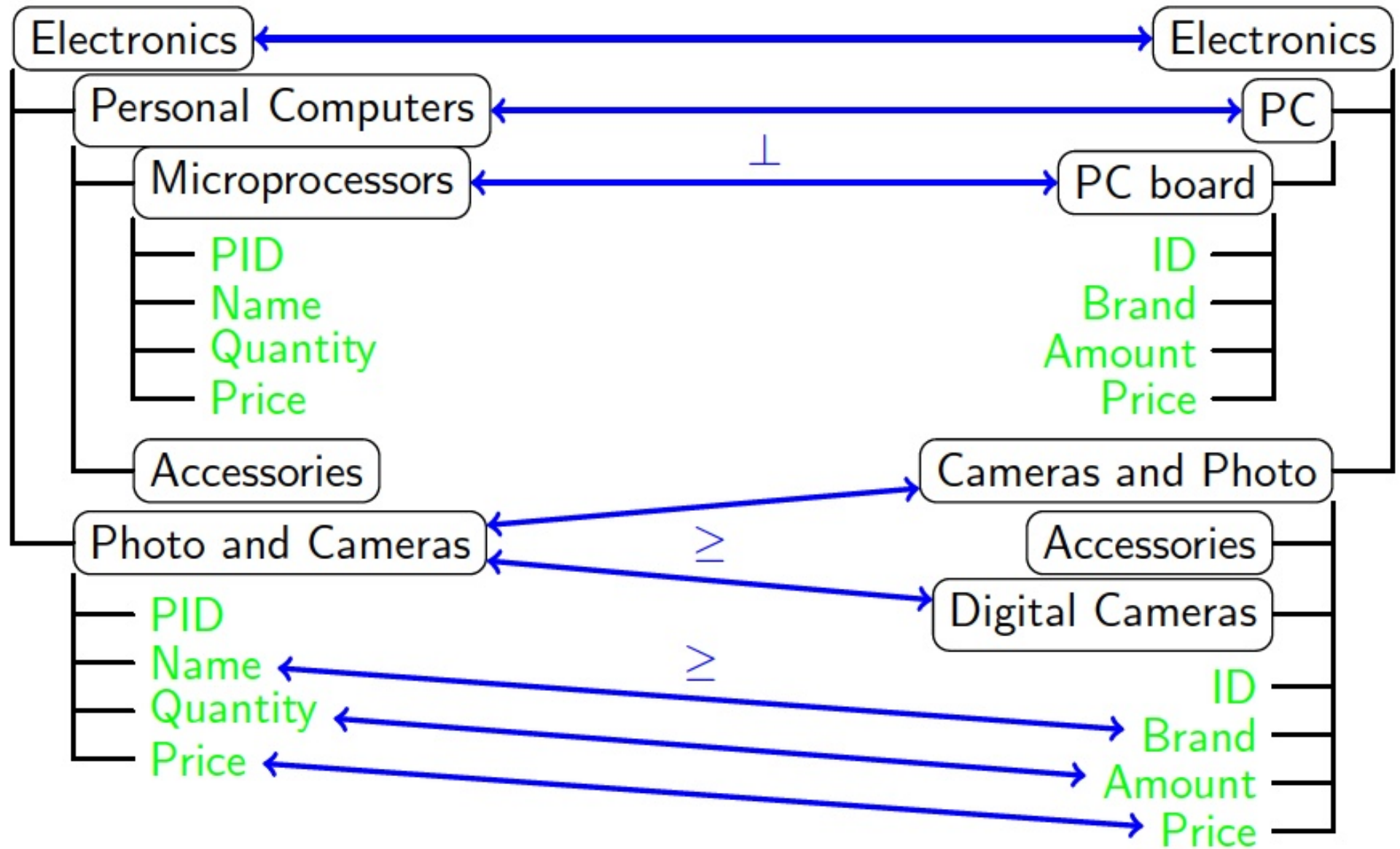
Problemă:

date la intrare ontologii – exprimate prin entități discrete (tabele, JSON, XML, clase, proprietăți,...) –, **trebuie determinate relațiile** (*e.g.*, echivalență, subsumare,...) care au loc între aceste entități

potrivirea ontologiilor

Problemă:
ontology matching

www.ontologymatching.org



Pavel Shvaiko & Jérôme Euzenat

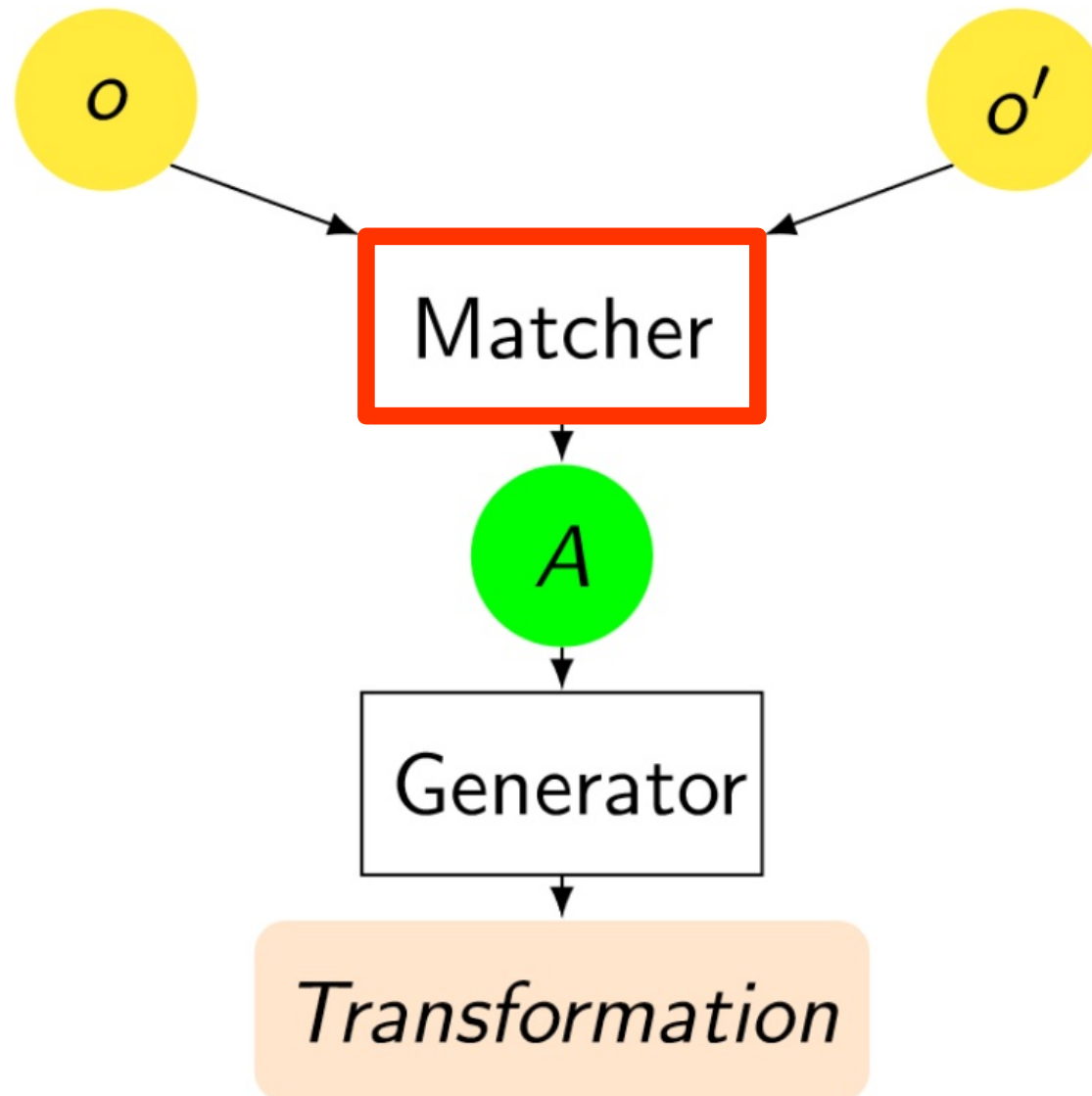


Pavel Shvaiko & Jérôme Euzenat

potrivirea ontologiilor

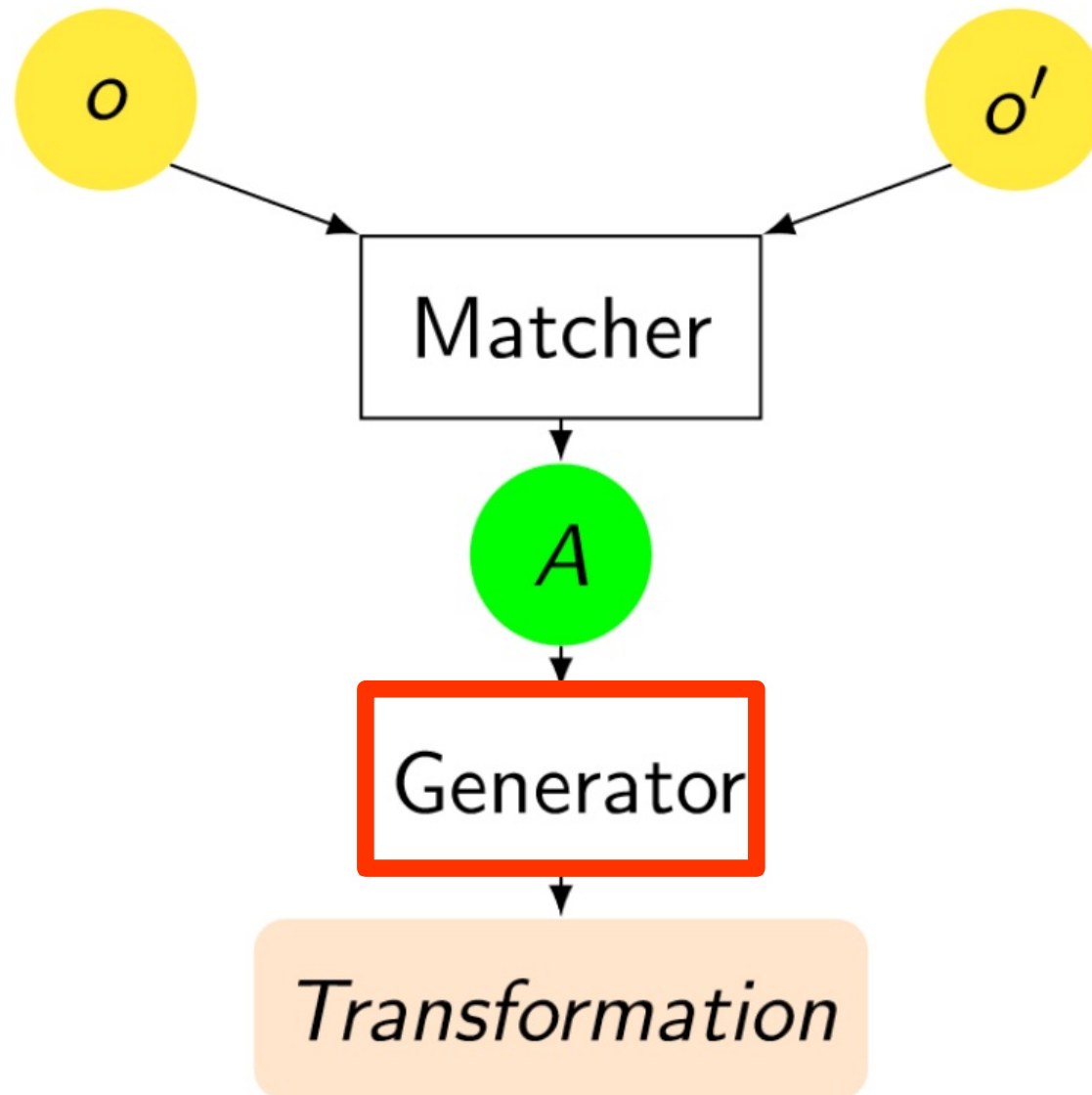
Reducerea eterogenității
poate fi realizată în 2 pași

1. potrivire (*match*) – determinarea alinierii



la momentul proiectării sau (re)utilizării ontologiei

2. procesarea alinierii



processe: fuzionare (*merging*), transformare etc.

potrivirea ontologiilor

Corespondența între O_1 și O_2

$$\langle id, e_1, e_2, r \rangle$$

id = identificator unic al corespondenței

e_i entitate din ontologia O_i (*e.g.*, clasă,...), $i=1,2$

r = relație între e_1 și e_2

echivalență ($\equiv$), disjuncție ($\perp$) etc.

potrivirea ontologiilor

Alinierea (*alignment*) A între O_1 și O_2 este o mulțime de corespondențe peste O_1 și O_2

prezintă o anumită cardinalitate: 1 la 1, 1 la N etc.

potrivirea ontologiilor

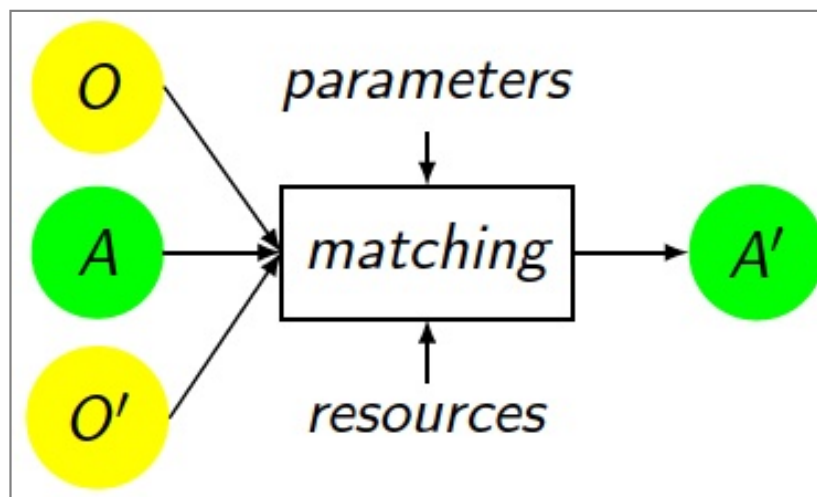
Alinierea (*alignment*) A între O_1 și O_2 este o mulțime de corespondențe peste O_1 și O_2

prezintă o anumită cardinalitate: 1 la 1, 1 la N etc.

pot fi atașate meta-date adiționale:
metoda de aliniere, data alinierii, proprietăți specifice,...

potrivirea ontologiilor

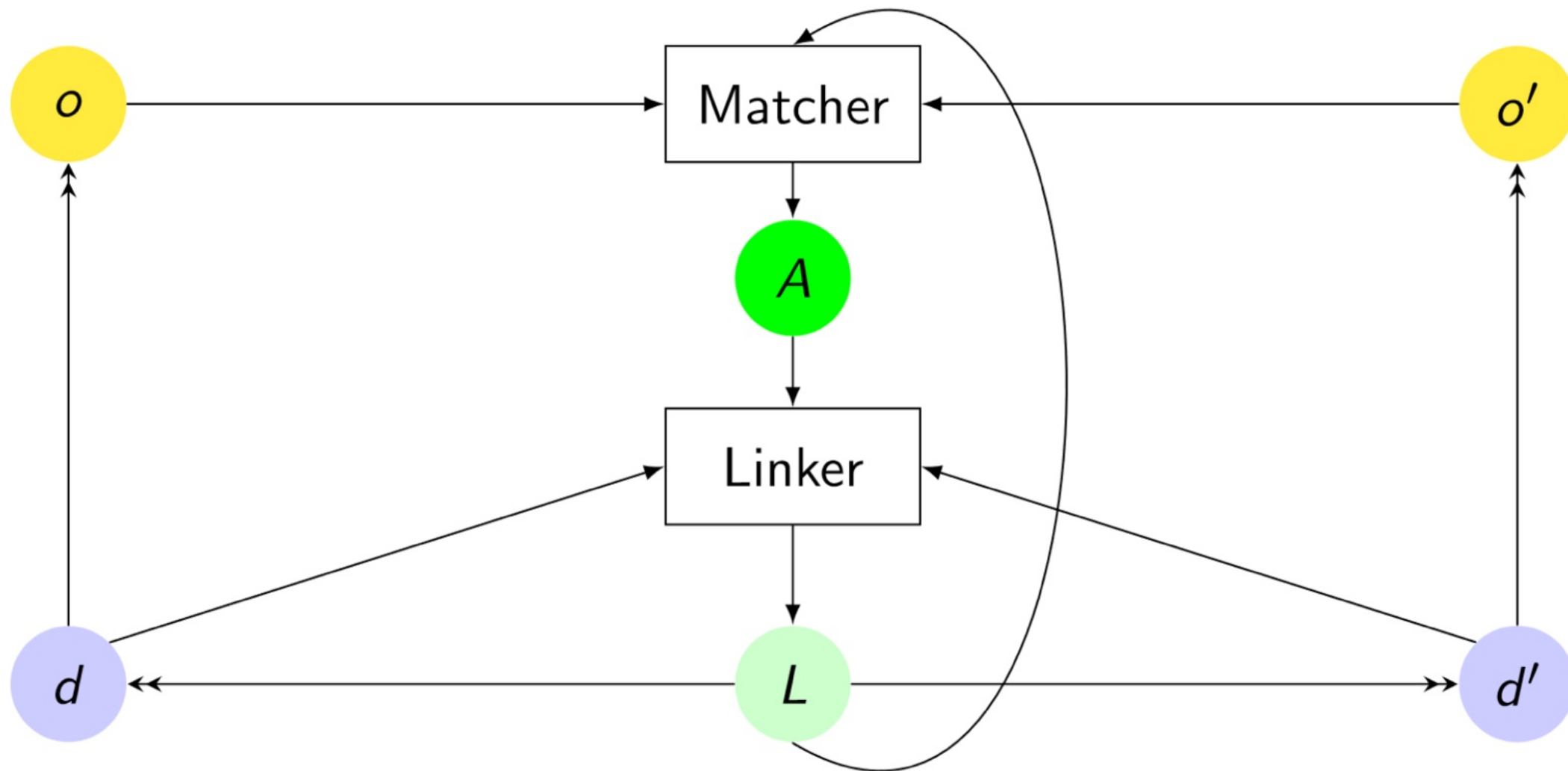
Alinierea (*alignment*)



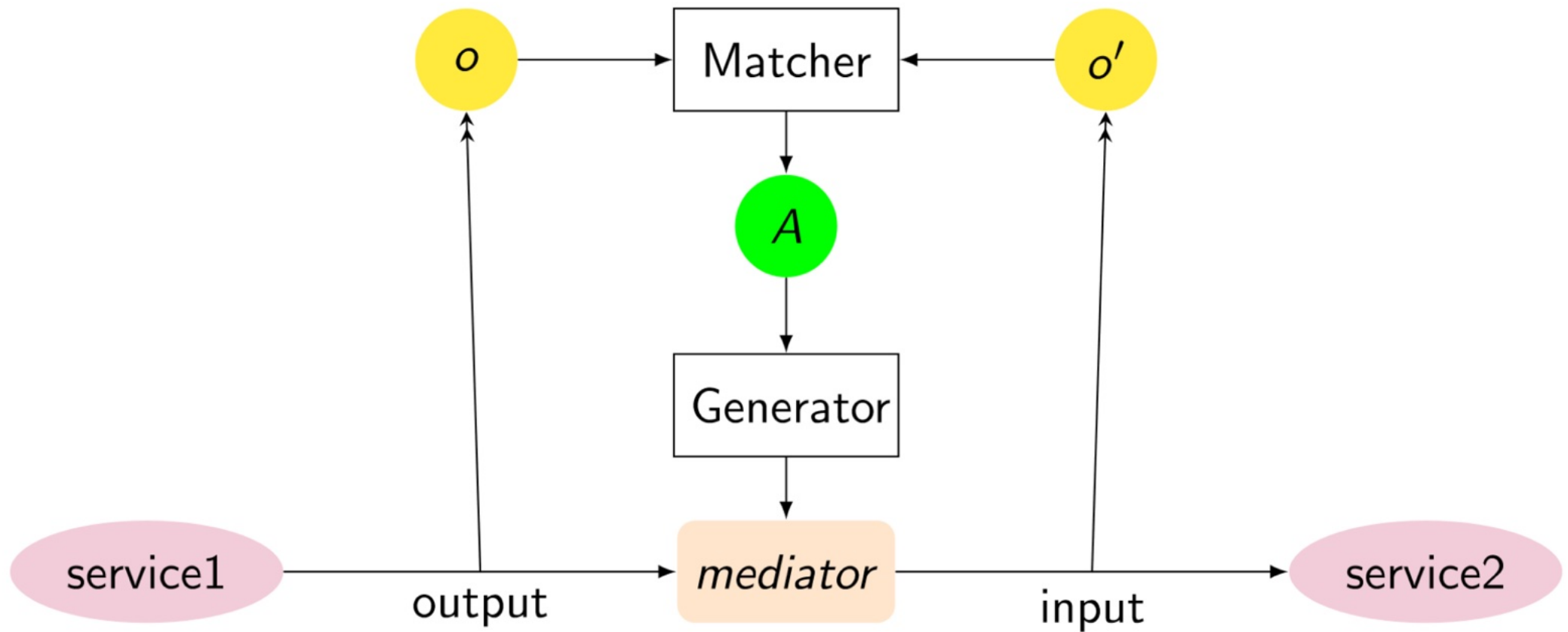
potrivirea ontologiilor: **utilizare**

Application	instances	run time	automatic	correct	complete	operation
Ontology evolution	✓			✓	✓	transformation
Schema integration	✓			✓	✓	merging
Catalog integration	✓			✓	✓	data translation
Data integration	✓			✓	✓	query answering
P2P information sharing		✓				query answering
Web service composition		✓	✓	✓		data mediation
Multi agent communication		✓	✓	✓	✓	data translation
Query answering	✓	✓				query reformulation

conform Jérôme Euzenat & Pavel Shvaiko



aspect de interes: **facilitarea interconectării datelor RDF**
linked data interlinking
(Jérôme Euzenat & Pavel Shvaiko)



aspect de interes: **compunerea serviciilor Web**
Web service composition
(Jérôme Euzenat & Pavel Shvaiko)

potrivirea ontologiilor: **utilizare**

Patrimoniul cultural (*cultural heritage*)

alinieri realizate în faza de proiectare a aplicațiilor de
integrare a capodoperelor descrise via lexicoane
precum *Iconclass* și *Aria*

alinierile rezultate pot fi folosite și pentru a facilita
căutările multicriteriale (*faceted search*)

potrivirea ontologiilor: **utilizare**

Geografie computațională (*geo-information*)

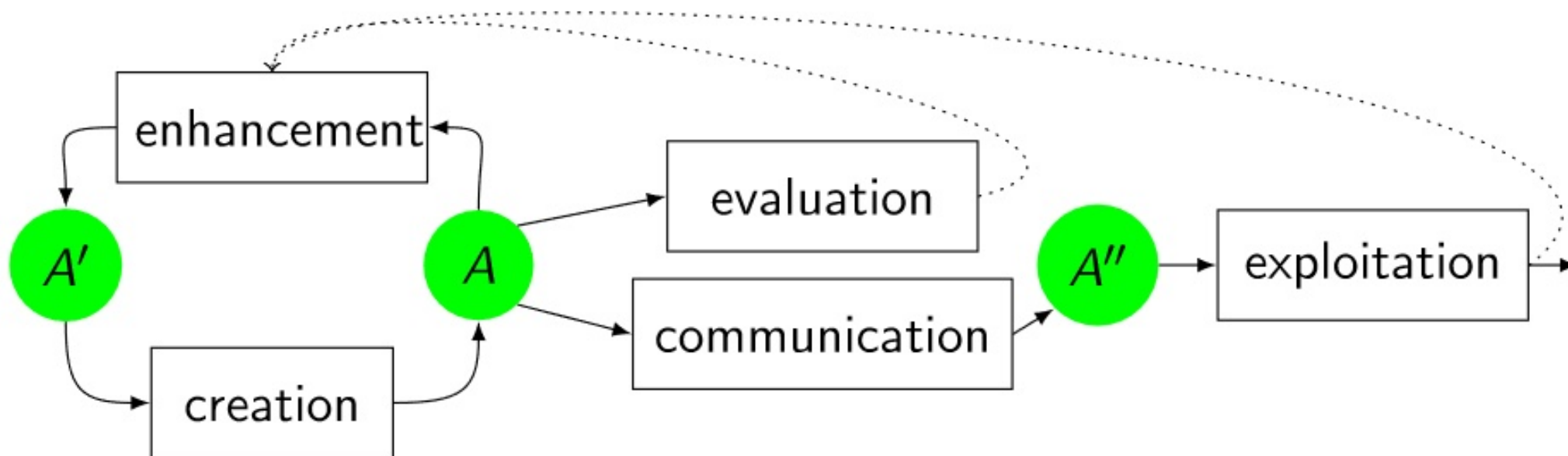
exemplu: planificarea urbană

bazată pe terminologii din arii diverse
(hidrologie, topografie) utilizate pentru a oferi
rezultate de interes conform cererilor utilizatorului

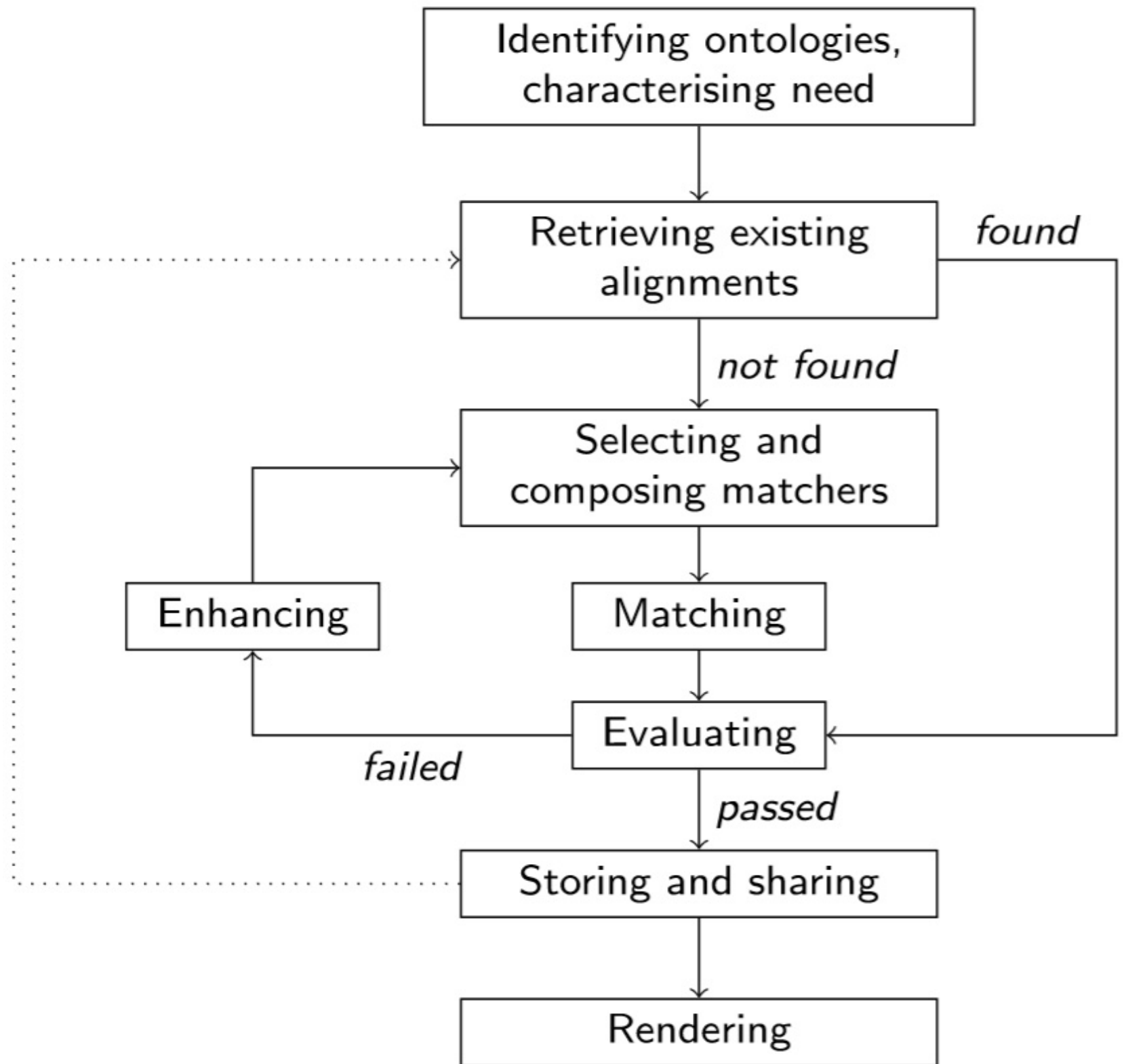
alinierile sunt (re)folosite la expandarea interogărilor

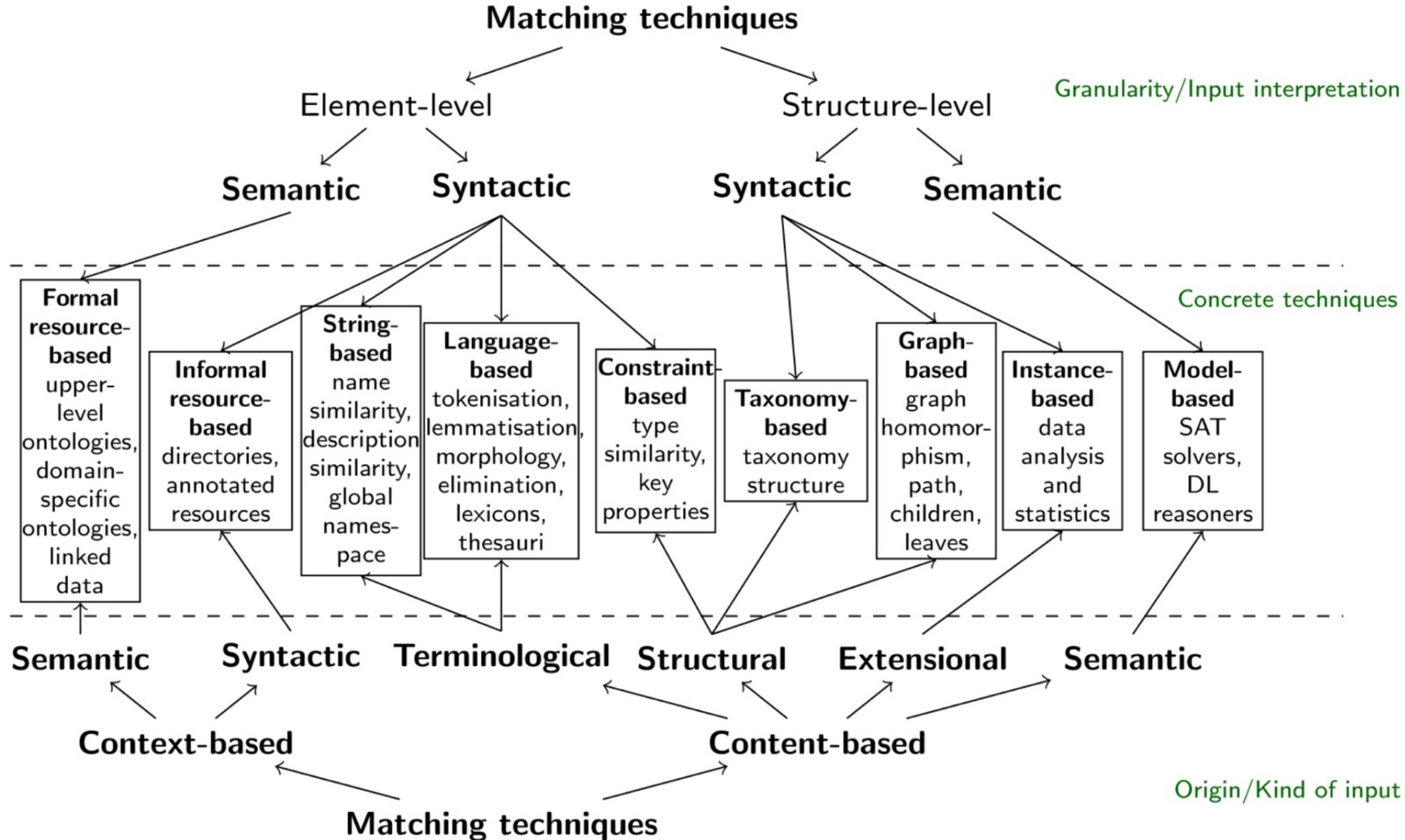
potrivirea ontologiilor

Alinierea – ciclul de viață



Alinierea etape



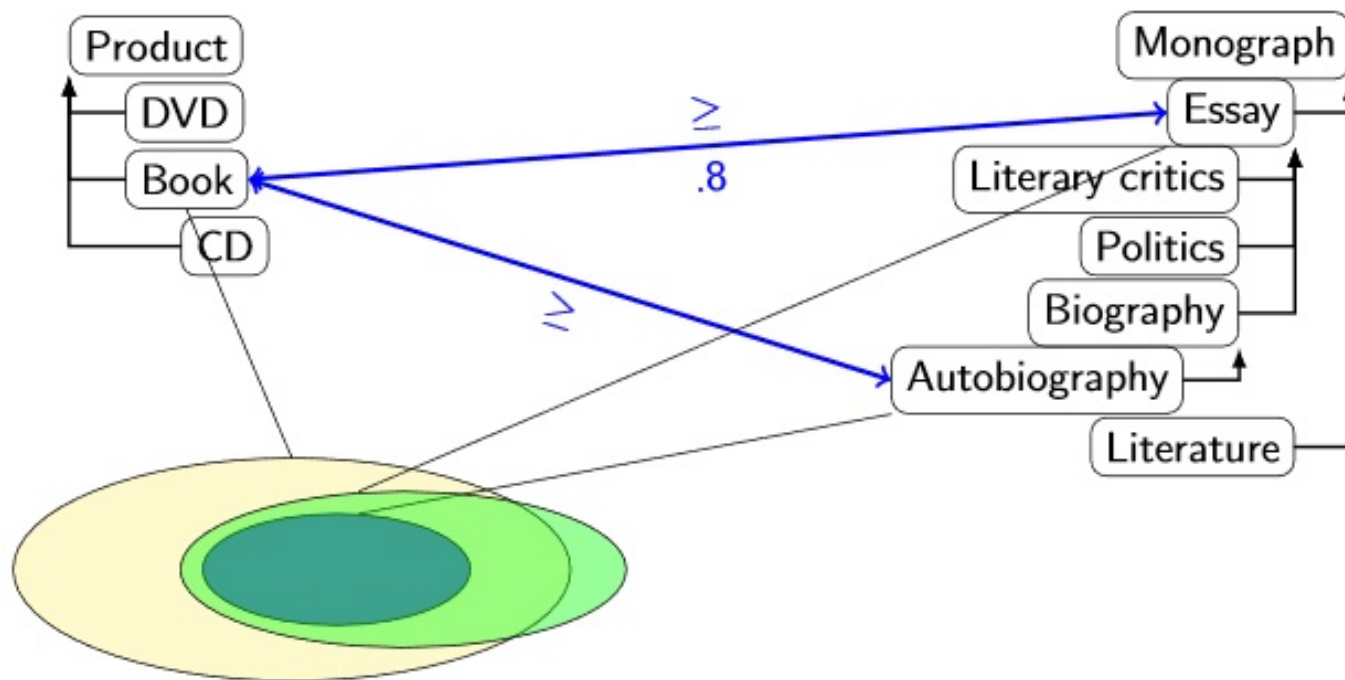


clasificarea tehnicilor de potrivire
(Euzenat & Shvaiko, 2014)

potrivirea ontologiilor

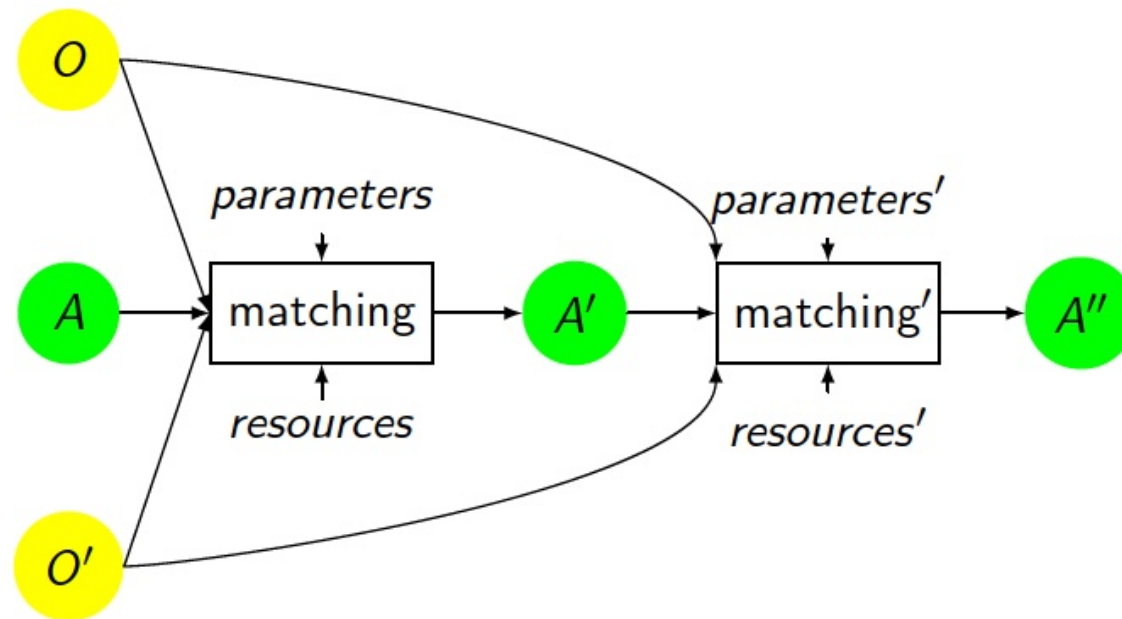
Extensional techniques

folosind o mulțime de instanțe vizând un concept,
se aplică metode statistice și/sau de învățare automată



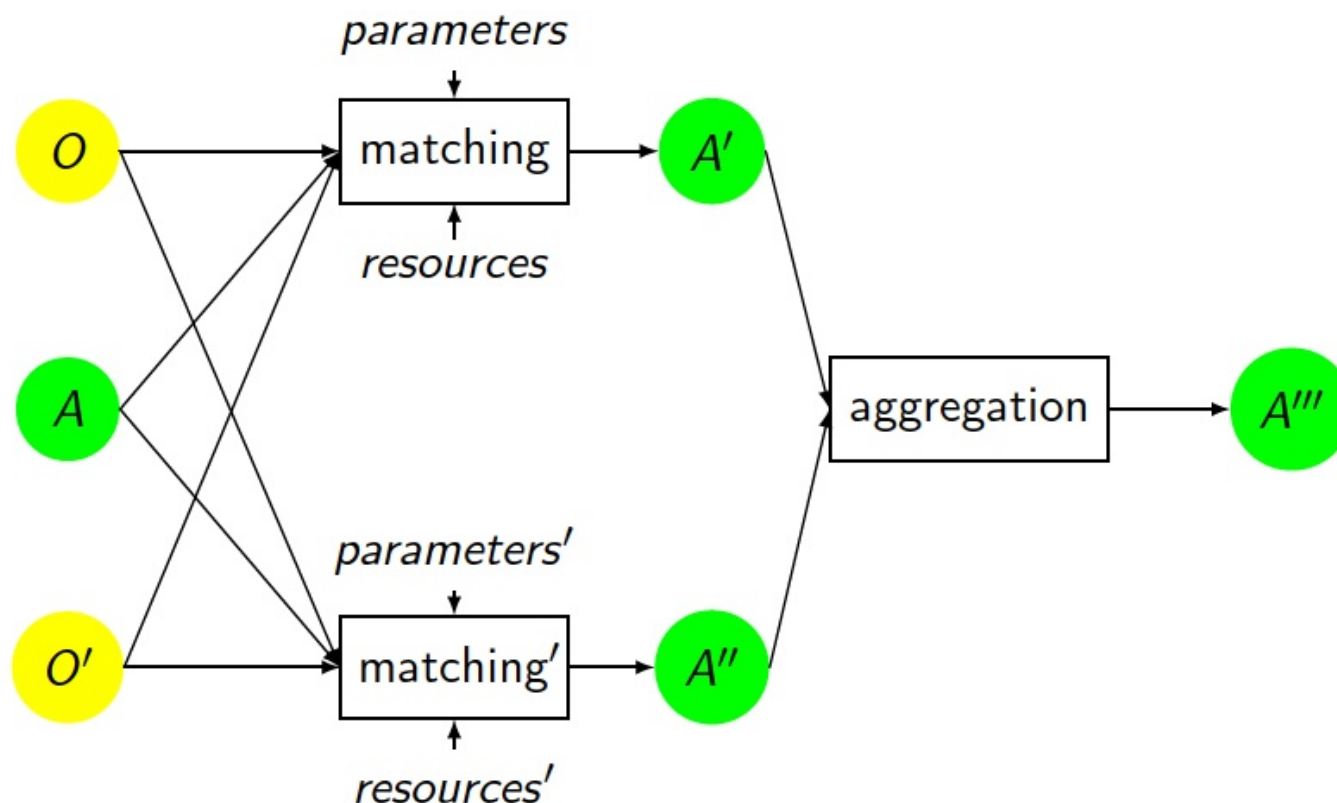
potrivirea ontologiilor

Procesele de potrivire pot avea loc **secvențial**



potrivirea ontologiilor

Procesele de potrivire se pot desfășura în paralel



potrivirea ontologiilor

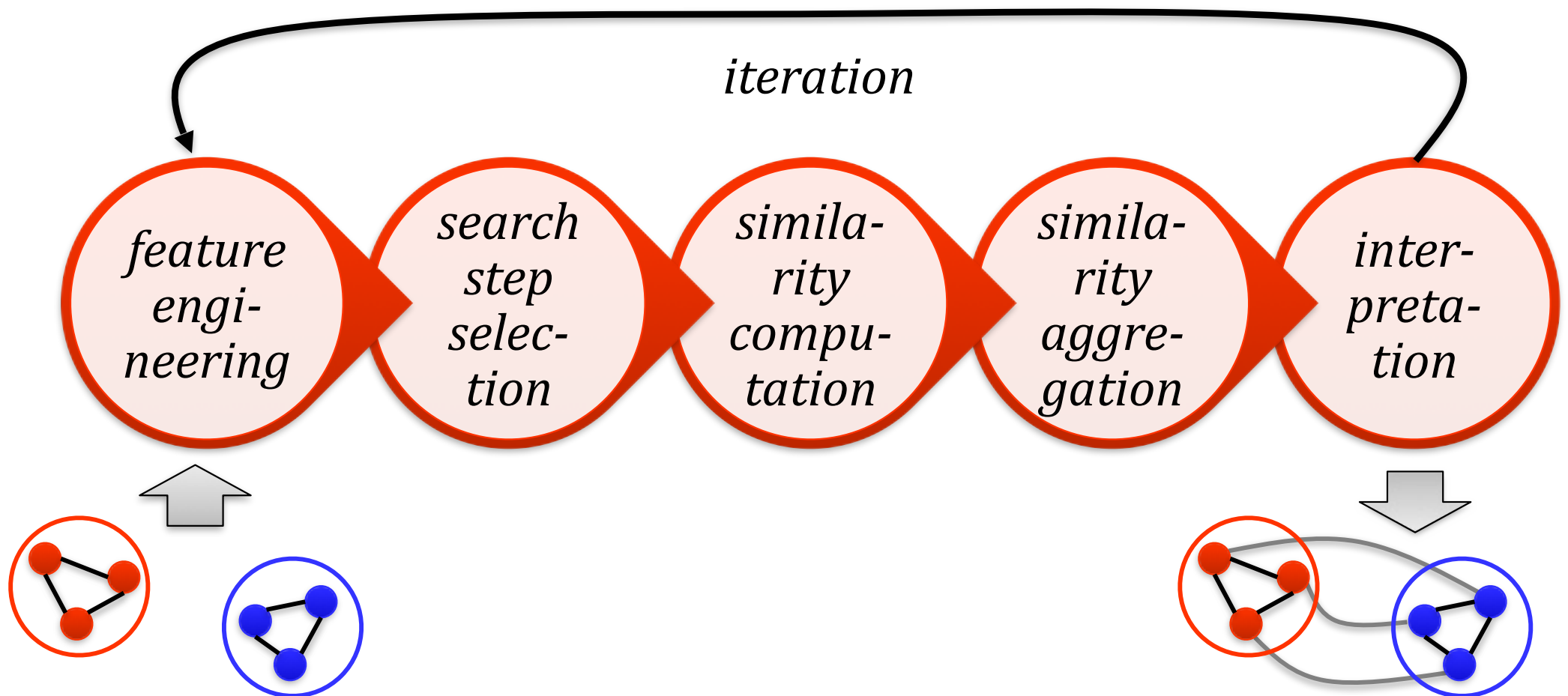
Remarcă:

*“Matching, however, requires **explanations** because mappings between terms are not always intuitively obvious to human users.”*

Pavel Shvaiko

aliniera ontologiilor

Procesul general: metodologie pentru crearea asocierilor – *mappings* (M. Ehrig, 2006)



alinierarea ontologiilor

Date de intrare:
două ontologii O_1 și O_2

eventual, pot fi puse la dispoziție asocieri cunoscute
(realizate manual, semi-automat ori automat)

alinierea ontologiilor

Date de ieșire:
o reprezentare a alinierilor
tabel indicând relația align_{O_1, O_2}

suplimentar, se pot oferi și valori de încredere

alinierea ontologiilor

Pasul 1: *feature engineering*

sunt selectate fragmente mici descriind entități de interes – reprezentând anumite semantici

ulterior, aceste trăsături vor fi folosite pentru compararea ontologiilor

alinierarea ontologiilor

Pasul 2: *search step selection*
se caută candidați pentru aliniere

se calculează similaritatea unor perechi
de concepte (c_1, c_2) , cu $c_1 \in O_1, c_2 \in O_2$

alinierarea ontologiilor

Pasul 2: *search step selection*
se caută candidați pentru aliniere

unele construcții pot fi ignorate

de exemplu,
se poate compara o1:processor cu o2:microprocessor,
dar nu neapărat cu o2:hasUSBPort

alinierarea ontologiilor

Pasul 2: *search step selection*
se caută candidați pentru aliniere

suplimentar, se pot adopta strategii (euristici) de
reducere a spațiului comparațiilor pentru ontologii largi

alinierea ontologiilor

Pasul 3: *similarity computation*

pentru descrierile date ale 2 entități din spațiul de candidați ai alinierii, se indică o similaritate

alinierea ontologiilor

Pasul 3: *similarity computation*

pentru descrierile date ale 2 entități din spațiul de candidați ai alinierii, se indică o similaritate

măsura similarității poate fi dată de una sau mai multe caracteristici (trăsături, *features*)

alinierarea ontologiilor

Pasul 3: *similarity computation*

exemplu:

$\text{sim}_{\text{label}}(\text{o1:processor}, \text{o2:microprocessor}) =$
 $\text{sim}_{\text{synonym}}(\text{"processor"}, \text{"CPU"}) = 1$

alinierarea ontologiilor

Pasul 4: *similarity aggregation*

pentru 2 entități, pot exista mai multe similarități,
conform criteriilor considerate

e.g., privind etichetele conceptelor sau
relațiile cu alte entități

alinierea ontologiilor

Pasul 4: *similarity aggregation*

pentru 2 entități, pot exista mai multe similarități,
conform criteriilor considerate

aceste valori se agregă
într-o unică valoare de similaritate

alinierarea ontologiilor

Pasul 4: *similarity aggregation*

exemplu:

$$\begin{aligned} &(\text{sim}_{\text{label}}(\text{o1:processor}, \text{o2:microprocessor}) + \\ &\text{sim}_{\text{subconcepts}}(\text{o1:processor}, \text{o2:microprocessor}) + \\ &\text{sim}_{\text{instances}}(\text{o1:processor}, \text{o2:microprocessor})) / 3 = 0.5 \end{aligned}$$

combinarea acestor valori se realizează
prin mai multe strategii

alinierarea ontologiilor

Pasul 5: *interpretation*

valorile de similaritate individuale și agregate
sunt utilizate pentru alinierea entităților

alinierarea ontologiilor

Pasul 5: *interpretation*

valorile de similaritate individuale și agregate
sunt utilizate pentru alinierea entităților

aceste similarități vor fi interpretate
(se aleg diverse criterii: prag limită, structură etc.)

alinierarea ontologiilor

Pasul 5: *interpretation*

la final, se oferă sau nu o schemă de aliniere a entităților considerate

exemplu: $\text{align(o1:microprocessor)} = \perp$



nu s-a putut determina
nicio aliniere convenabilă

alinierea ontologiilor

Pasul 6: *iteration*

similaritatea e propagată în ontologii
pe baza relațiilor între entități

alinierarea ontologiilor

Pasul 6: *iteration*

similaritatea e propagată în ontologii
pe baza relațiilor între entități

la fiecare interacție, va fi recalculată similaritatea
unei alinieri candidate
pe baza similarității perechilor entităților vecine

alinierea ontologiilor

Pasul 6: *iteration*

după realizarea unei interații,
pot rezulta alte similarități

alinierea ontologiilor

Pasul 6: *iteration*
procesul se termină
atunci când nu se propun noi alinieri

alinierarea ontologiilor

Abordările avansate de aliniere utilizează:

potrivire contextuală (CBM – *context-based matching*)

potrivire bazată pe învățare (*matching learning*)

metode bazate pe reguli

tehnici de analiză structurală a grafurilor

alinierarea ontologiilor

Abordările avansate de aliniere utilizează:

tehnici de raționament – *e.g., theorem proving*

ajustare optimală (*tuning*)

îmbunătățirea alinierii: *quality measures, debugging*

algoritmi combinați, cu implicarea expertului uman
(*user-driven composition, collective matching*)

alinierarea ontologiilor

Există mai mulți algoritmi propuși

competiția anuală OAC – *Ontology Alignment Contest*
(2004—2025)

oaei.ontologymatching.org

detalii în P. Shvaiko, J. Euzenat, “Ontology Matching:
State of the Art and Future Challenges”,
IEEE Transactions on Knowledge and Data Engineering (2013)
disi.unitn.it/~p2p/RelatedWork/Matching/SurveyOMtkde_SE.pdf

alinierarea ontologiilor

Problematici – începând cu 2019:
potriviri la nivel conceptual (*TBox/schema matching*)

e.g., în domeniul bioinformaticii + e-medicinei
Adult Mouse Anatomy (2744 clase) vs. *NCI Thesaurus* (3304 clase)
Biodiversity & Ecology, Disease & Phenotype,
Large Biomedical Ontologies, Food Nutritional Composition
evaluarea performanței alinierii datelor multilingve
7 ontologies of the Conference domain translated into 8 languages

alinierarea ontologiilor

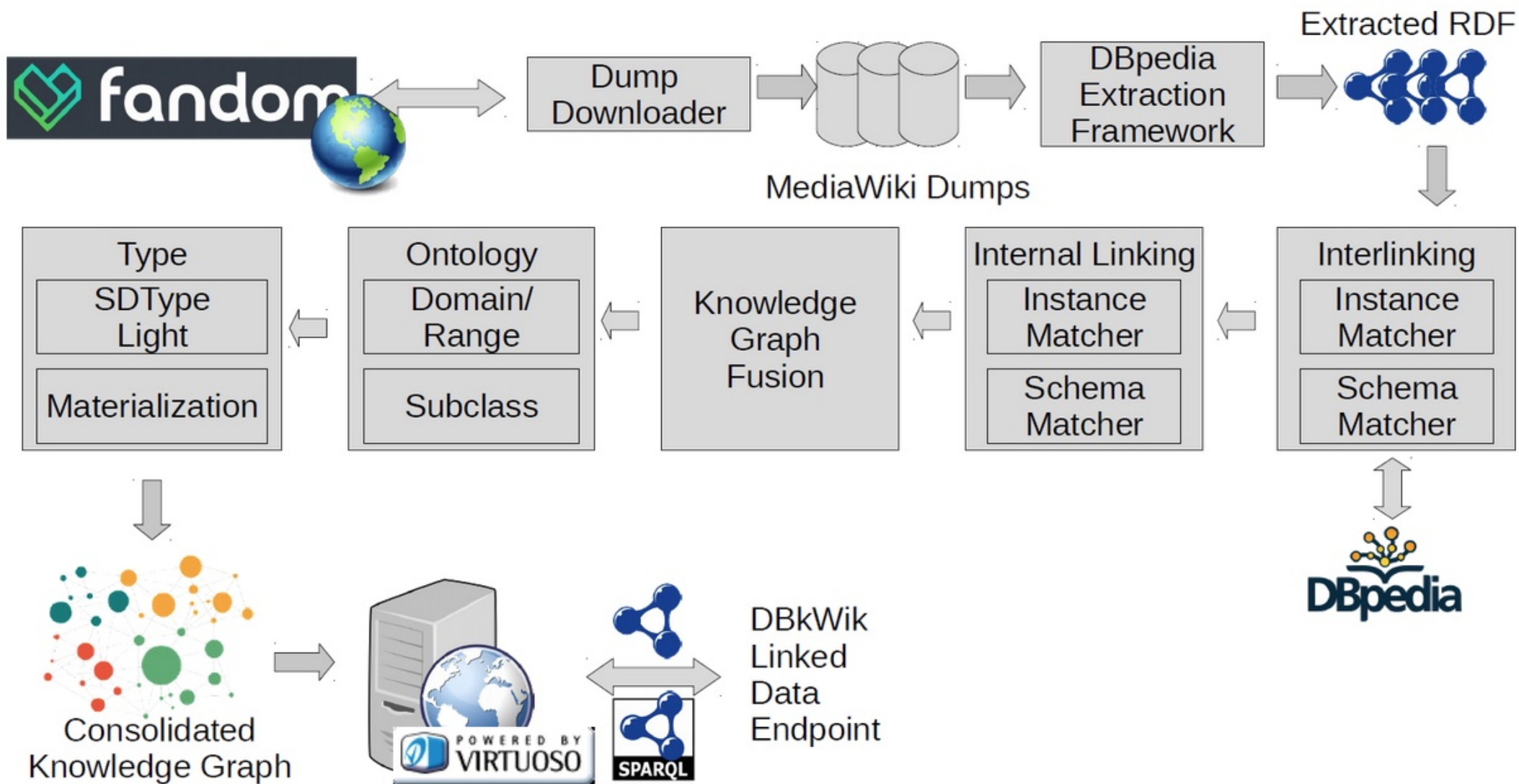
Problematici – 2019, 2021:

potriviri la nivel de instanțe + scheme (*match **both** the instances and the schema from multiple knowledge graphs*)

exemplu: cunoștințe despre producții cinematografice preluate
– via instrumentul **DBkWik** –

de la grupuri de *wiki*-uri de tip *fandom* (*wikifarms*) găzduite de Wikia
oaei.ontologymatching.org/2021/knowledgegraph/

Source	Source URL	Language	Hub	Topic	#Instances	#Properties	#Classes	Dump
Star Wars Wiki	http://starwars.wikia.com	en	Movies	Entertainment	145,033	700	269	rdf/xml
The Old Republic Wiki	http://swtor.wikia.com	en	Games	Gaming	4,180	368	101	rdf/xml
Star Wars Galaxies Wiki	http://swg.wikia.com	en	Games	Gaming	9,634	148	67	rdf/xml
Marvel Database	http://marvel.wikia.com	en	Comics	Comics	210,996	139	186	rdf/xml
Marvel Cinematic Universe Wiki	http://marvelcinematicuniverse.wikia.com	en	Movies	Entertainment	17,187	147	55	rdf/xml
Memory Alpha	http://memory-alpha.wikia.com	en	TV	Entertainment	45,828	325	181	rdf/xml
Star Trek Expanded Universe	http://stexpanded.wikia.com	en	TV	Entertainment	13,426	202	283	rdf/xml
Memory Beta	http://memory-beta.wikia.com	en	Books	Entertainment	51,323	423	240	rdf/xml



procesarea datelor/cunoștințelor
dbkwik.webdatacommons.org

alinierarea ontologiilor

De asemenea, se pot folosi cunoștințe anterioare
(*background knowledge*)

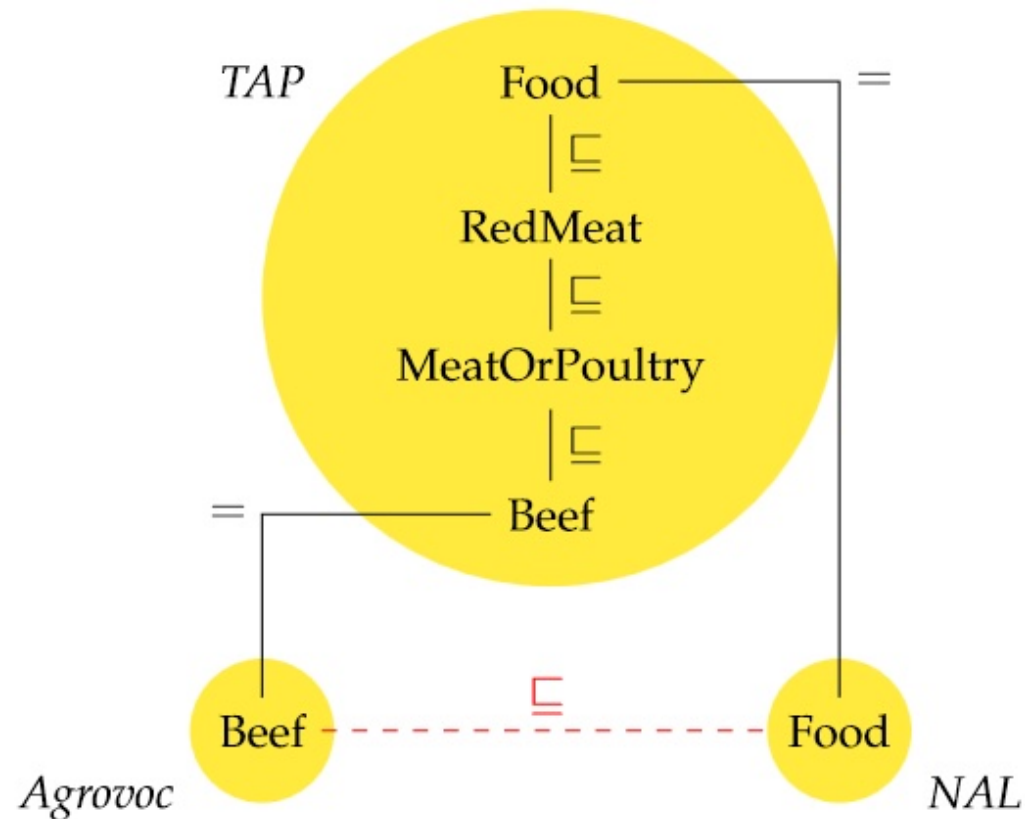
ontologii de referință ale domeniului

ontologii de nivel superior – *e.g.*, DOLCE, KKO, SUMO

resurse textuale *on-line* (Wikipedia – DBpedia, Wikidata)

vocabulare/ontologii disponibile *on-line*

WordNet



pentru două entități provenind din vocabularele AGROVOC și NAL
(*National Agricultural Library*) se deduce automat
că termenul *Food* este mai general decât *Beef*

sistemul de efectuare a potrivirilor (*matcher*) a recurs la cunoștințe
anterioare (*background knowledge*) disponibile pe Web

alinierarea ontologiilor: dezvoltare

Software de referință (Java):
Alignment API and Alignment Server
moex.gitlabpages.inria.fr/alignapi/

formate RDF folosite:

Alignment format – tradițional (doar XML)

+

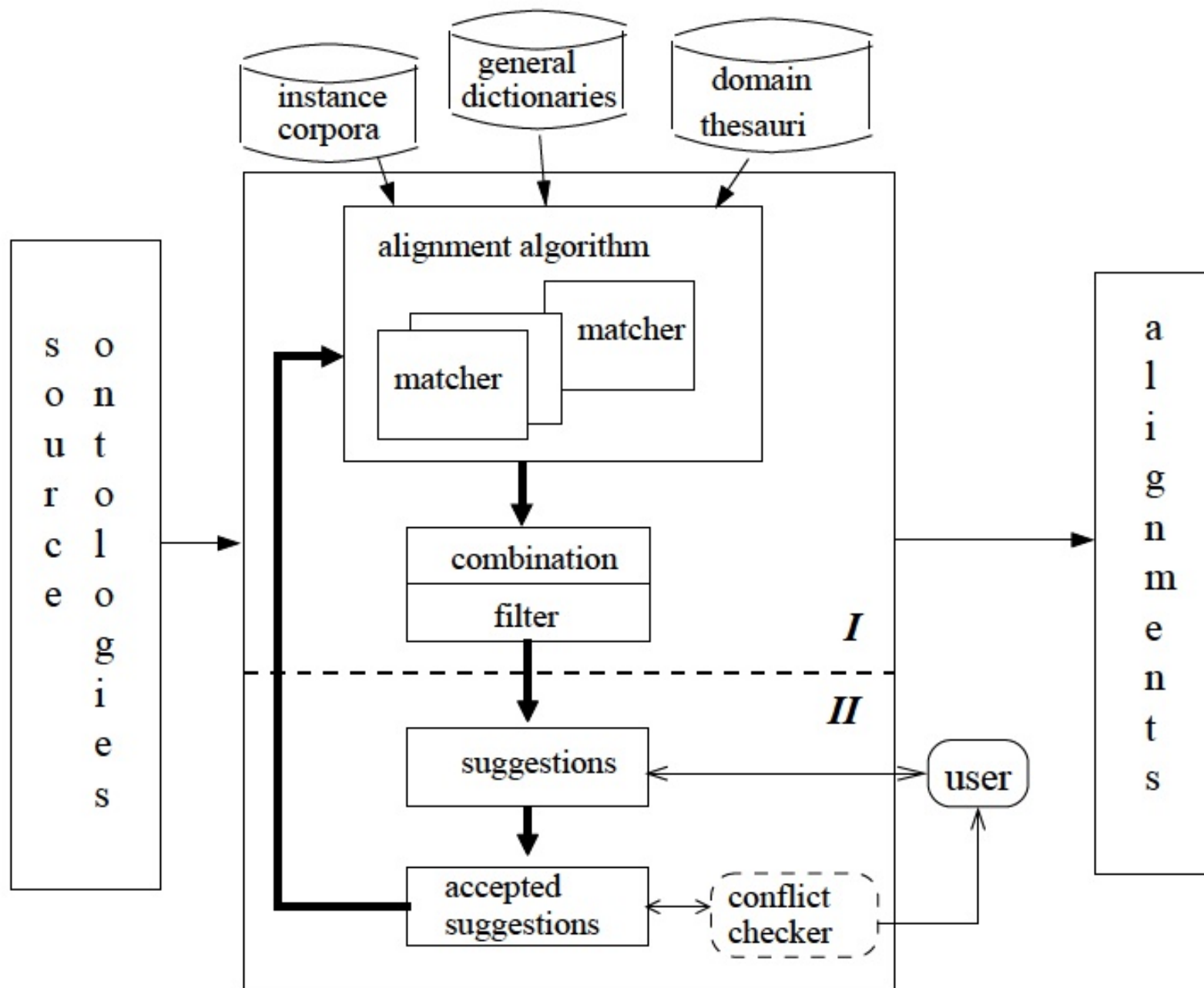
Expressive and Declarative Ontology Alignment Language
mai expresiv, recurge la sintaxele XML și N3

alinierea ontologiilor: dezvoltare

```
// corespondențe (de tip Cell) dintre 2 entități (clase EDOAL) a 2 ontologii  
O1:entity1 a edoal:Class .  
O2:entity2 a edoal:Class .
```

```
align:map [ a align:Cell ;  
  align:entity1 O1:entity1 ;  
  align:entity2 O2:entity2 ;  
  align:measure "1.0"^^xsd:float ;  
  align:relation "=" ;  
  edoal:transformation "..."  
] .  
...
```

asociere (**map**) bazată pe o relație (**relation**) – aici, de echivalență – cu un grad de încredere (**measure**) via un algoritm de transformare (**transformation**)



proces de bază vizând alinierea (Lambrix & Tan, 2005)

implementări: **Alignment**

Platformă Web *open-source* colaborativă

alinierile generate sunt disponibile via SPARQL
sau un API tradițional

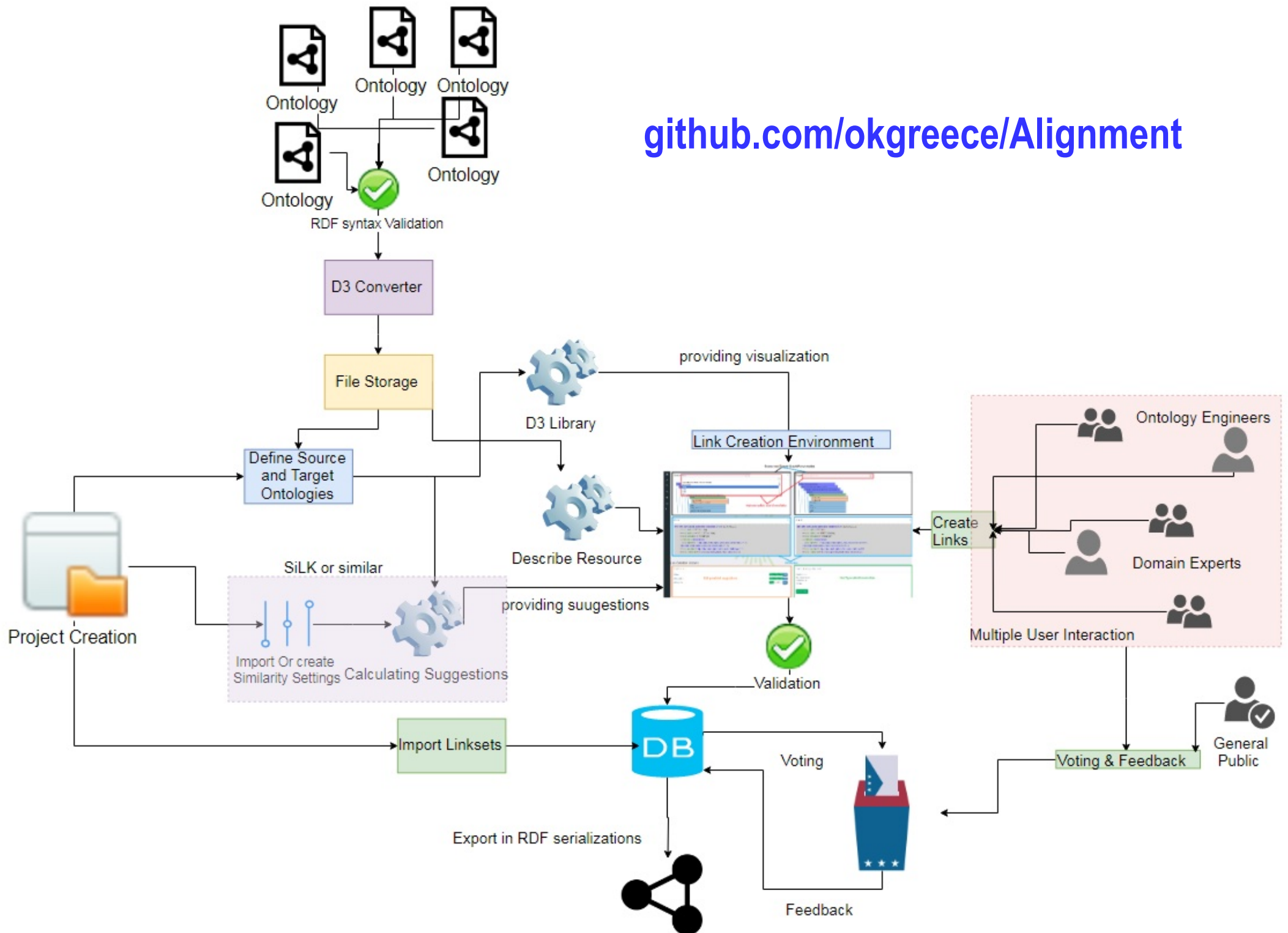
implementare PHP + Java

detalii în S. Karampatakis *et al.*, *Alignment: A Collaborative, System Aided, Interactive Ontology Matching Platform*, OM 2017

disi.unitn.it/~pavel/om2017/papers/om2017_poster3.pdf

alignment.okfn.gr

github.com/okgreece/Alignment



AgreementMaker – include o multitudine de sisteme automate de potrivire, cu suport pentru extensii

ASMOV – integrarea informațiilor din aria bioinformaticii

CogZ – investighează interacțiunea cu utilizatorul în cadrul proceselor de aliniere ontologică

eTuner – *metamatcher*, capabil să ajusteze automat diverși parametri de control

Falcon – utilizează *divide-et-impera* și algoritmi de *clustering* pentru alinieri de ontologii largi

SAMBO – focalizat asupra potrivirii și fuziunii de ontologii biomedicale

RiMOM – *framework* ce folosește strategii multiple conform unor algoritmi de decizie bayesiană

utilizarea alinierilor

După efectuarea alinierilor, pot avea loc procese precum:

fuziune – **Merge** $(o, o', A) = o''$

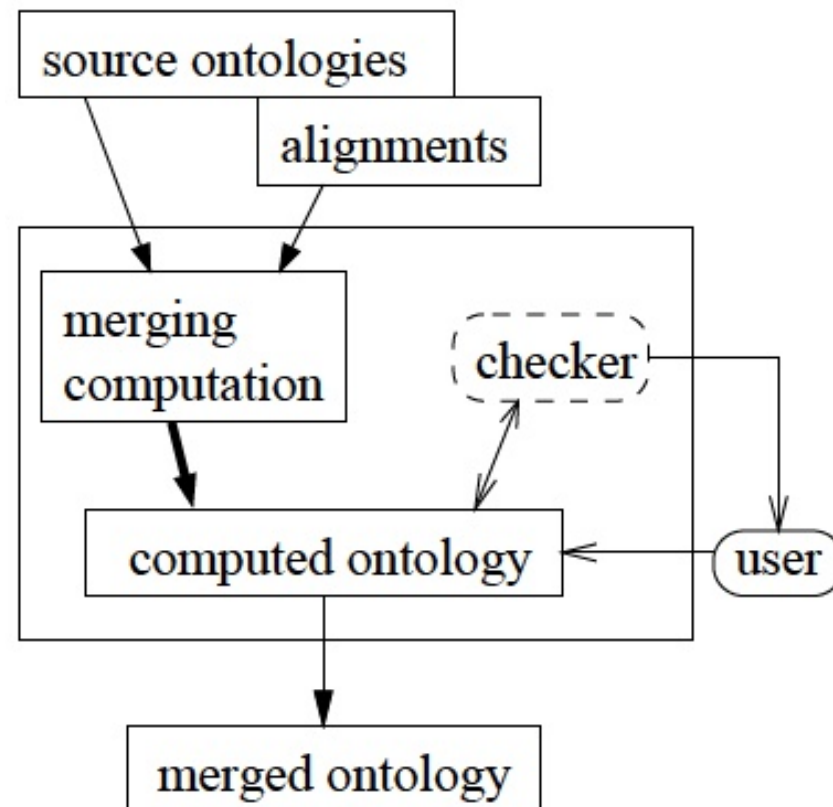
traducere – **Translate** $(d, A) = d'$

interconectare – **Interlink** $(d, d', A) = L$

rescriere de interogări – **TransformQuery** $(q, A) = q'$

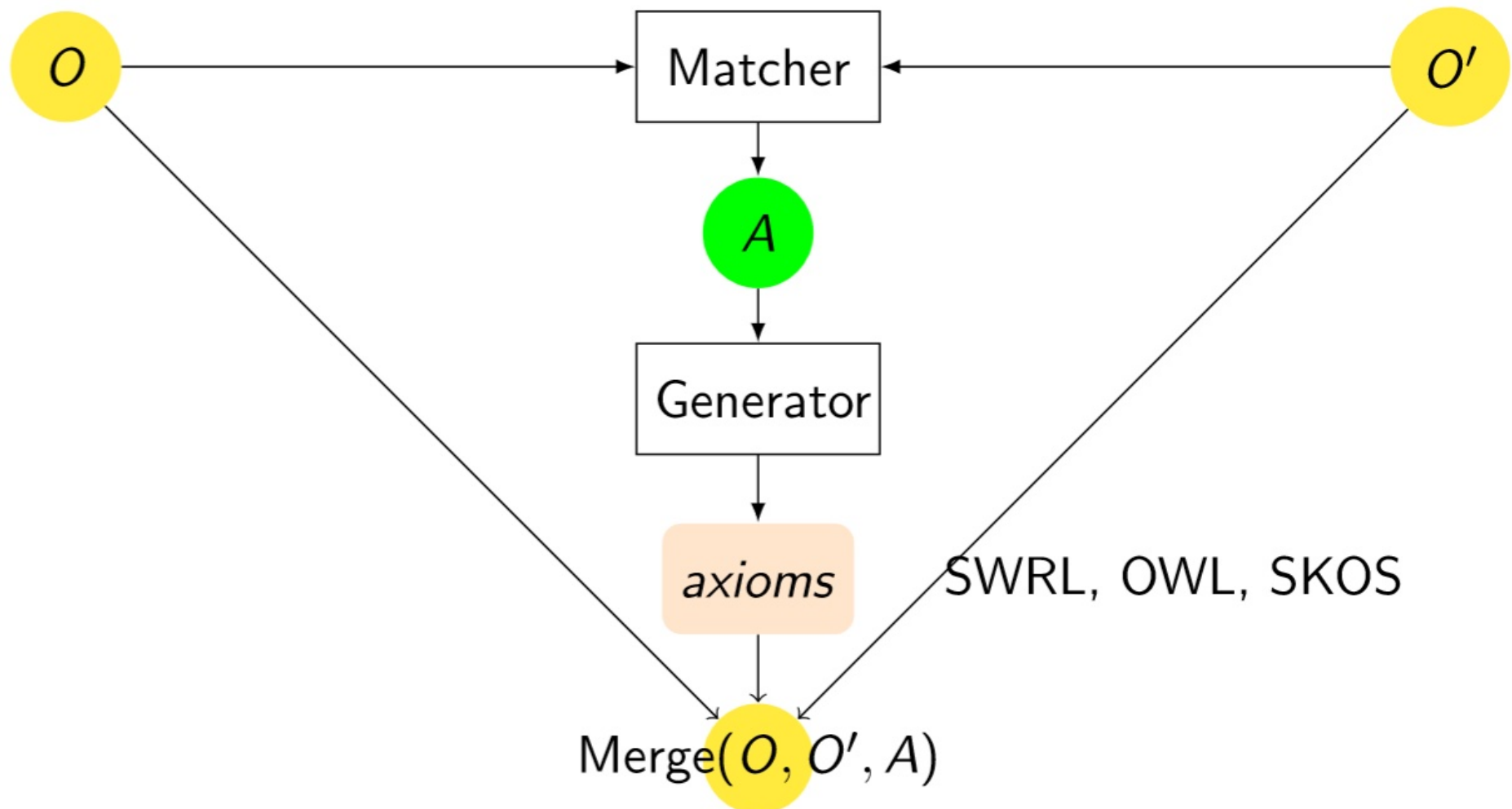
fuziunea ontologiilor

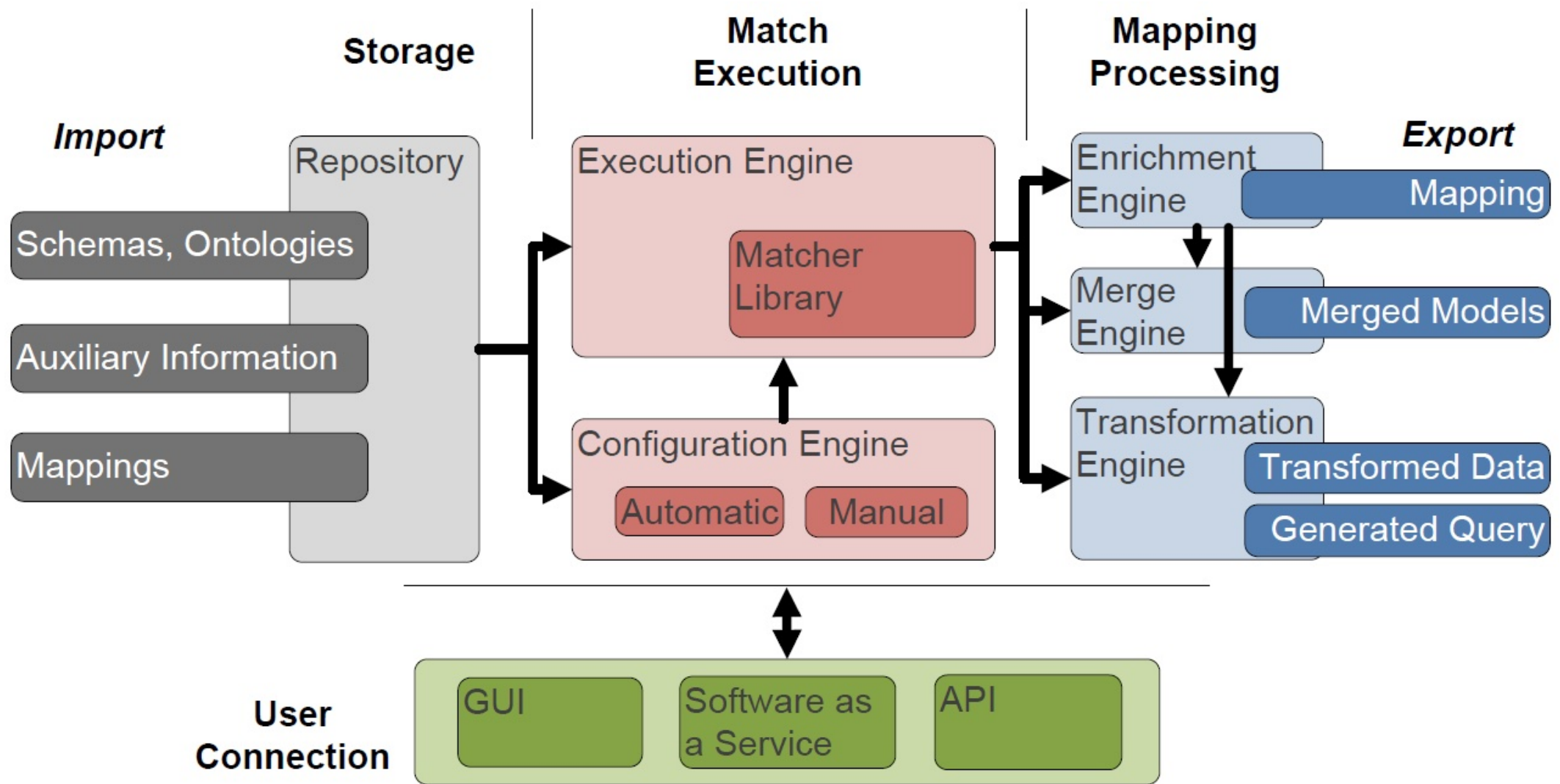
Merging ontology – algoritmul general:



fuziunea ontologiilor

Merging ontology – în cazul ontologiilor OWL:





arhitectura generală a unui sistem de realizare
a potrivirilor și fuziunilor dintre ontologii

„concluzii”

*“An **exemplary ontology** is one that can serve as a model that can be imitated or leveraged by ontology engineers in the future. The most fundamental property of an exemplary ontology is being **well designed for its intended purpose.**”*

ontologydesignpatterns.org/wiki/Odp:Exemplary_ontology

rezumat



knowledge engineering
metodologii, șabloane + alinierea ontologiilor